FishBot v0.4: Exact Public-Belief Inference and Declaration as Optimal Stopping in Six-Player Canadian Fish

A rules-exact engine, a tractable exact posterior, and a duplicate-block evaluation protocol

FishLab Research Project

August 21, 2026

Abstract

We present FishBot v0.4, an agent for the six-player, 54-card Canadian Fish variant with nine six-card half-suits and exact teammate-allocation declarations. The work rests on a structural observation: because every card movement in Fish is publicly observed, a card whose location has never been revealed is still with whoever was dealt it, so the entire hidden state of the game is the initial deal. Inference therefore reduces to a degree-constrained counting problem whose layer index is determined by its own state, and we solve it exactly — exact ownership marginals, the exact joint probability of a named half-suit allocation, and exact deal sampling — in a few hundred thousand arithmetic operations. The one disjunctive constraint, the certificate that an ask carries about the asker's own hand, is confined to a single half-suit and is handled exactly by enumerating that half-suit rather than by approximation; ablations show it is by a wide margin the single most valuable thing the agent knows. A one-line consequence of ask legality gives a strategic result that no prior Fish agent implements: a half-suit held entirely by one team can never be asked in by the other, so it can never be taken back, so waiting to claim it carries no risk. Declaration becomes an optimal-stopping problem rather than a confidence threshold — though our ablations locate the credit in the exact allocation probability rather than in the stopping rule that consumes it, and we say so. The same theorem has a consequence we did not anticipate: with every opponent void in every live half-suit, no ask can move a card, and two policies that both correctly refuse to cash an uncertain half-suit will cycle rather than finish, so the rules as usually stated do not guarantee termination against deterministic play. We also correct four rule omissions in the predecessor simulator, most importantly that a claim is legal at any moment. On an untouched seed bank played in six-rotation duplicate blocks, FishBot v0.4 beats FishBot v0.3 75.07% of the time (95% CI 73.71–76.40, a cluster bootstrap over deals) and wins at least 75.07% against every policy in the population — the worst case, not the average, because the question the project asks is whether one policy can be best across playstyles. Its declaration forecasts are calibrated to 0.0222 expected calibration error, against 0.1257 for its predecessor, which is what makes the accuracy gap between them a fact about estimation rather than about nerve. An adversary fitted specifically to beat the frozen agent, inside the same policy class and with the same optimiser, fails to establish any advantage over it, while the same procedure beats FishBot v0.3 comfortably — a lower bound on exploitability rather than a bound, but the comparison is the point. We are precise about the one place approximation remains: the posterior is exact with respect to the rules, not with respect to opponents' playstyles.

1 Introduction

Canadian Fish — Literature in most of the published literature — is a six-player, two-team card game of imperfect information. On your turn you name a card and an opponent; if they hold it you take it

and continue, and if they do not, the turn is theirs. At any moment either team may claim a half-suit by naming which teammate holds each of its six cards, and any error in that claim hands the half-suit to the opponents. Everything that happens is public.

The game has almost no computational literature. There is no arXiv paper on it, no engine, and one open-source learning agent that plays a different variant. That absence is not because the game is easy: it has six players, two teams that cannot communicate, an information set of astronomical size, and a horizon of sixty to a hundred and twenty decisions. It is closer to Bridge or Skat than to anything solved.

FishBot v0.3 [1] was the first serious attempt: a transparent one-ply utility policy over an approximate posterior, fitted on 126,600 simulated games and evaluated on held-out seeds. It established that remembering public asks and reconciling them against card counts dominate everything else, and it left three things open — an approximate belief model, a confidence threshold in place of a declaration policy, and a simulator that did not implement the rule allowing a claim at any moment.

This paper closes all three. The route runs through one structural observation.

> *Cards in Fish move only through publicly observed transfers. Therefore a card whose location has never been revealed is still with whoever was dealt it, and the entire hidden state of the game is the initial deal.*

The consequences are unusually strong. Inference becomes a degree-constrained counting problem over a capacity vector, which we solve exactly — exact marginals, exact probabilities for a named allocation, and rejection-free sampling — for a few hundred thousand arithmetic operations (§5). The one constraint that resists a product form, the certificate that an ask carries about the asker's own hand, is confined to a single half-suit and can therefore be enumerated inside it exactly rather than approximated. And a one-line reading of ask legality yields a strategic result that no previous agent implements: a half-suit held entirely by one team can never be asked in by the other, so it can never be taken back, so waiting to claim it is free (§6). Declaration stops being a threshold and becomes an optimal-stopping problem.

Claims

This paper defends four bounded claims.

1. **The exact posterior is computable and is computed.** Belief marginals, the probability of a named allocation, and consistent deal samples are exact with respect to every deduction the rules license, and the implementation is validated against brute force, against an independent dynamic program, and against exact rejection sampling.
2. **A half-suit held entirely by one team cannot be taken back** (Theorem 2), so declaration is optimal stopping rather than a threshold. The agent that follows declares near-perfectly; we are explicit in §11 that the ablations credit the exact allocation probability rather than the stopping rule itself.
3. **FishBot v0.4 is the strongest policy in the tested population**, and — the point of the exercise — it is strongest against *every* member of that population, not merely on average.
4. **The forecasts it stops on are calibrated**, which is what makes the third claim a statement about decision quality rather than about one seed bank.
5. **It is much harder to counter than its predecessor.** An adversary fitted specifically to beat it, in the same policy class and with the same optimiser and budget, does not reach 50% against it; the identical procedure beats FishBot v0.3 comfortably. This is a lower bound on exploitability rather than a bound, and we say so, but the comparison between the two figures is the meaningful part.

A sixth result was not among the claims we set out to make and is reported because it is a fact about the game rather than about the agent: because a half-suit frozen by Theorem 2 also admits no further

information about itself, two policies that both correctly decline to cash an uncertain one deadlock permanently, and the rules as usually stated therefore do not guarantee termination against deterministic play (§6.3).

This paper does not claim a Nash or team equilibrium, an exploitability bound, or that Fish is solved. §14 is explicit about what remains, including the one caveat that most constrains the word "exact": the posterior is exact with respect to the *rules*, and a fully Bayesian agent would additionally weight deals by how likely the observed actions are under each opponent's particular playstyle. We implement that correction, fit it, and report that it adds nothing once the rule-based certificates are enforced — which is evidence about this population rather than a general theorem.

2 The game, and the rule dialect studied

Canadian Fish (also called Literature) is played by six players in two teams of three, seated so that the teams alternate. The deck is the standard 52 cards plus two distinguishable jokers, partitioned into nine *half-suits* of six cards: the low band $\{2, 3, 4, 5, 6, 7\}$ and the high band $\{9, 10, J, Q, K, A\}$ of each of the four suits, together with a ninth half-suit containing the four eights and the two jokers. Every player is dealt nine cards. The team that claims at least five of the nine half-suits wins.

Asking. On their turn a player asks one opponent for one named card. The ask is legal only if the asker holds at least one *other* card of that card's half-suit, does not hold the named card, and the target still has cards. If the target holds the card they must surrender it and the asker continues; if not, the turn passes to the target. Every ask, its answer, and every resulting transfer is public, as is every player's card count.

Declaring. A player may declare a half-suit *at any moment*, including during an opponent's turn, and needs to hold none of its cards to do so. The declaration must name, for each of the six cards, the exact teammate who holds it. If every card and every assignment is right, the declaring team scores the half-suit; *any* error awards it to the opposing team. Either way the six cards leave the game, and the turn returns to whoever held it.

Running out. A player with no cards drops out of the asking cycle: they can neither ask nor be asked. If the turn reaches them — which can only happen after a declaration — they hand it to a teammate who still has cards, and the team may negotiate openly, but may communicate nothing beyond willingness to receive. If an entire team is cardless, the other team must declare every remaining half-suit with no further asking, again choosing the declarer by open negotiation limited to willingness.

2.1 What the v0.3 simulator modelled, and what it did not

The v0.3 engine [1] implemented the deck, the ask legality conditions, transfers, exact-allocation declarations and the cardless-team endgame, and it is the direct ancestor of this work. Comparing it against the authoritative rules above surfaces four gaps, all of which change strategy rather than merely presentation:

1. **Declarations were restricted to the declarer's own turn.** The real rule permits declaring at any moment. This matters because it turns declaration timing into a continuous decision rather than a once-per-turn one, and because it allows a team to cash a half-suit immediately before an opponent could act.

2. **The cardless player's choice of successor was fixed** to the teammate holding the most cards, rather than being a decision.
3. **The forced endgame selected the declarer by comparing private confidences**, which leaks more than the rules permit; the rules allow the team to exchange only willingness.
4. **Games were adjudicated by majority holding** after an action cap. A cap is a necessary safety device — §6.3 shows the rules do not guarantee termination against deterministic play — but adjudication changes the payoff structure of exactly the stalling behaviour that strong policies produce. v0.4 makes termination a policy property instead, and retains a cap only as a backstop whose residue is split neutrally by majority holding and whose incidence is reported with every experiment.

The v0.4 engine implements all four correctly and exposes each as a switch, so that the effect of the rule dialect is measurable rather than assumed (§12). Unless stated otherwise, every number in this paper uses the full rules: out-of-turn declarations enabled, cardless players allowed to declare, willingness-only endgame negotiation, and a 400-ask safety cap whose incidence we report (360 under the v0.3 dialect, matching that engine).

3 Related work

3.1 Fish and Literature specifically

The literature on this game is human and informal. McLeod's Pagat entry [2] is canonical for the rules — including our 54-card nine-set deck and the variant in which any declaration error awards the half-suit to the opponents — and for a tactics list covering deliberate misses that inform a partner, locking out a dangerous opponent by never asking them anything, and exhausting one rank across all opponents before switching rank; it also records Ali Salahuddin's partner-signalling convention and Guy Srinivasan's Forced Claims rule. Develin's chapter [3] is the deepest strategy writing we found and describes exactly our dialect: the blackball example, the rule against declaring a half-suit one holds entirely, the observation that asking teaches opponents as much as teammates — so asks inside a team-owned half-suit are a free channel — and the corpus's only quantitative endgame, where the alternative to a coin-flip declaration would need success probability 3/2. He forbids pre-agreed conventions outright, contradicting Pagat. Wikipedia [6] names the "stalemate breaker"; secondary pages [4, 5] restate blackballing, asking the asker, and an endgame delegation heuristic.

Computational prior art is close to absent. The review corpus reports that arXiv searches on 21 August 2026 for `"Canadian Fish"`, `"Literature card game"` and `"half-suit"` each returned nothing, that a GitHub sweep found no repository containing search, planning or a working learner, and that Literature is not among RLCard's environments [10]. The one real agent is Somani's `literature` [7, 8]: about 1,940 lines of Python with a complete rules engine and a sound deduction fixed point. It plays the 48-card, eight-half-suit "remove the sevens" variant and trains only on four players, so the six-player game's hardest sub-problem is missing — with one teammate a half-suit allocation spans 2^6 possibilities and is usually forced, with two it spans $3^6 = 729$. Its learner, a $1149 \to 100 \to 1$ perceptron fitted for 1,030,945 iterations, never bootstraps: it regresses a hand-written score of ± 20 per ask outcome plus ± 100 at the end, which punishes the deliberate miss both Pagat and Develin recommend, and its move generator forbids known misses except as a fallback. A verified bug compounds this — team identity is an integer and the winner a plain enumeration member, so the terminal comparison is always false and every move in every game received -100. The published model never saw a win/loss gradient and is functionally a greedy maximum-likelihood asker. Declarations fire the instant a half-suit is provably owned, wrong claims are unpenalised by construction, and no strength numbers have ever been published. The other public repositories are user interfaces; one closed-source Android application advertises four- and six-player bots with no stated methodology (unverified) [9].

3.2 Search under imperfect information

The standard recipe for hidden-hand card games is determinization, or Perfect Information Monte Carlo: sample an assignment of the hidden cards consistent with the public history, solve the resulting perfect-information game with a double-dummy solver, and play the best average move. Levy proposed it for Bridge (a source the corpus could not verify) [11]; Ginsberg's GIB made it work, reweighting sampled deals by Bayes and finding fifty worlds sufficient [12]. Frank and Basin showed the method unsound however many worlds are drawn, naming *strategy fusion* — the searcher plays a different strategy in each world, although a real player must commit to one per information set — and *non-locality*, where a node's value depends on parts of the tree outside its subtree because informed opponents steer play away from it [13]; the repairs that work are those attacking non-locality too [14]. Long, Sturtevant, Buro and Furtak explained when determinization succeeds at all, through leaf correlation, bias and a disambiguation factor, placing Skat and Hearts where PIMC is near-optimal and Kuhn poker where it is not [15]. Ginsberg names the structural cost: because each determinization assumes the missing information will arrive at the next decision, PIMC never makes an information-gathering play.

Information Set MCTS instead builds one tree over information sets, redeterminizes each iteration, and explores on the count of iterations in which a move was *available* rather than on parent visits [17]. Its variants coincide when all moves are fully observable, as in Fish, and one tree of ten thousand iterations beat forty trees of two hundred and fifty by 22.9%; re-determinization at non-root seats and self-determinization extend it [20, 18], though ISMCTS is unsound too, its exploitability sometimes *increasing* with computation [21]. The Skat and Bridge engineering lines matter as much: Kermit's inference module alone was worth +217 tournament points per 36 games [23], and a Spades post-mortem traced blunders to a sampler placing a card of true probability 1/3 into 87% of its worlds [19].

None of this is the right foundation here. In Fish the *perfect-information* game is degenerate: a clairvoyant player only ever names cards the target actually holds, so never fails an ask and never loses the turn, and declares each half-suit correctly the instant their team completes it. The double-dummy value of almost every position is "the team on turn takes everything it can reach"; the solver barely discriminates among root moves, and averaging over sampled worlds leaves

$$\arg\max_{(t,c)} \mathbb{E}[V^{\mathrm{PI}}] \;\approx\; \arg\max_{(t,c)} \Pr[\,t \text{ holds } c \mid h\,],$$

a one-ply heuristic blind to the information an ask leaks, the information a refusal supplies, the option value of postponing a declaration, and partner signalling. A Fish double-dummy solver will be easy to write, fast, and worthless as an engine — useful only for hit-probability features and as a sparring baseline. The corpus's reading of [15] sharpens the point: Fish should have a disambiguation factor far above the ≈ 0.6 measured for Skat and Hearts — nominally PIMC's best regime — yet its pre-terminal leaf structure sits in the corner where almost every move wins in the sampled world, which that paper never measured and dismissed as "very boring games". Fish is not boring, precisely because of the imperfect information determinization discards. That is why we build an exact belief engine (§5) and a decision-theoretic declaration rule (§6) instead. One smaller finding survives: Go Fish sits at the maximum of Zuckerman, Felner and Kraus's Opponent Impact measure [22], so which opponent to ask is a first-class argument here, not a tie-break.

3.3 Equilibrium computation and team games

The equilibrium line runs from counterfactual regret minimisation [27] through sampling variants [28], CFR+ [29] and discounted CFR [30] to function approximation [31, 32]. CFR+ does relatively badly where some actions are very costly mistakes [30], and a wrong declaration is exactly that; DREAM's exploration term is exponential in depth, and a Fish game runs sixty to a hundred and twenty decisions.

ReBeL recasts an imperfect-information game as a continuous-state perfect-information game over *public belief states* [33], building on the common-information reduction of decentralised control [35] and the public-belief construction used in Hanabi [43]; Student of Games extends it and names Scotland Yard as the closest published public-observation domain [34]. Fish is an *exact* public-belief-state game, which is what §5 exploits.

Three teammates share a payoff but hold different information and cannot communicate, so the team is not one player with pooled knowledge — assuming otherwise is a second flavour of strategy fusion — and Nash equilibrium over individual strategies is not the target either. The right concept is the team-maxmin equilibrium with correlation (TMECor): agree on a correlated joint strategy before the deal, then execute it without communicating. Celli and Gatti give the complexity — TMECor is FNP-hard, the team best-response oracle APX-hard, and the worst-case price of uncorrelation grows with the leaf count $|L|$, with reported bounds of $|L|/2$, $|L|/4$ and $\sqrt{|L|}$ [36]. Farina and coauthors connect ex-ante team collusion to extensive-form correlation [37], and the team belief DAG and two-player conversion make the team a single coordinator issuing prescriptions, giving a two-player zero-sum game whose equilibria are realization-equivalent to TMECor, at the cost of a representation exponential in how much private information distinguishes teammates within a public state [38, 39]. Learning-based attacks exist [40, 41]; the latter proves self-play converges to *local* team equilibria — the formal version of the complaint that a bot's asks stop meaning anything.

We attempt none of this: one seat's information set at deal time holds $45!/(9!)^5 \approx 1.9 \times 10^{28}$ deals, and a prescription would have to name an action for each teammate at each of their possible hands. The cost should be stated plainly. Nothing here is an equilibrium result — win rates are measured against a fixed population, robustness is a worst-case-over-panel claim rather than an exploitability bound, and the fitted exploiter of §9 is a lower-bound proxy whose failure mode is known: a low score against an exploiter is not evidence of low exploitability [71].

3.4 Cooperative play through public actions

Hanabi is the reference benchmark for coordination through observable actions [42], and is relevant because Fish likewise has no side channel. The Bayesian Action Decoder made the public belief explicit, reached 24.174 of 25, and attributed roughly 40% of its information transfer to learned conventions rather than grounded content [43]. The Simplified Action Decoder showed that letting teammates condition on the *greedy* rather than the exploratory action is a prerequisite for conventions to survive ε-greedy training [44]. Other-Play exposed how much of a self-play score is convention rather than skill: the same agents scored 23.97 with themselves and 2.52 paired across independent runs [45]. Off-Belief Learning is an explicit dial on convention depth, from a grounded level-one policy near 20.9 to 24.10 at level four, with a uniqueness result that makes independent runs interoperable [46]; test-time search alone lifted a blueprint from 24.08 to 24.61 [47]. Bridge bidding is the other model system, an auction that is literally a learned language: bidding networks beat a strong commercial program by 0.25 IMP per board [53], joint policy search — which scores simultaneous changes at several information sets, exactly what inventing a convention requires — improved a system from +0.29 to +0.63 IMPs per board [54], and an agent trained with human partners gained +0.97 IMPs per deal [55].

The asymmetry separating Fish from both is the audience. Hanabi has no adversary, so every bit of signal is pure profit — which is why its conventions are so elaborate — and in bridge the harm a leaked auction does is diffuse. In Fish a leaked card location is directly executable: once it is public that a player holds a card, any opponent holding another card of that half-suit takes it with probability one and keeps the turn. Conventions here carry a genuine eavesdropping cost. The corpus states the sharpest version information-theoretically: because teammate and opponent hands are exchangeable given the public history, an absolute codebook — one whose meaning is fixed by public information alone — is decoded

identically by all five listeners, so with two teammates against three opponents the Wyner-style secrecy rate of such a convention is non-positive [50, 51]; only a receiver-relative codebook, whose semantics depend on the receiver's own holdings, has positive rate, at the price of ambiguity at the encoder. We report this as an argument, not a theorem: the corpus gives a structural claim with an informal exchangeability justification and no proof, and we have not proved it either. It is the design rationale for building v0.4 with no private channel at all. Public-signalling economics predicts the same outcome, cheap talk separating before two audiences only when credible against both and otherwise collapsing toward pooling — the regime Farrell and Gibbons call subversion [49]. The Hanabi construction that most clearly fails to port is the modular hat-guessing code, which works because each receiver sees every other receiver's hand and can subtract it [48]; in Fish nobody sees any hand. Any claim that an agent has learned a convention must also survive the scramble ablation of [52], where high speaker–listener consistency coexisted with zero causal effect.

3.5 Inference over deals

Counting the deals consistent with a public history is a permanent computation. Ryser's inclusion–exclusion and Glynn's sign-vector formula are exact at $O(2^n n)$ [56, 57]; at $n = 54$ that is about 1.8×10^{16}, so they serve only as unit tests on tiny instances. The Jerrum–Sinclair–Vigoda chain is a fully polynomial randomised approximation scheme [58], but a practical study measured it at 50,634 seconds for $n = 10$ against Ryser's 0.003 seconds [59]. Matrix scaling is the usual approximation — Sinkhorn iteration, with the deterministic strongly polynomial analysis bounding the permanent within a factor e^n [60] — and it is what our fast path uses (§5.5); the corpus measures its cost as two to six percent maximum marginal error on a Fish-sized instance, worst exactly where a real position gets hard, reaching 5.9% at 60% void. The contingency-table literature contributes sequential importance sampling [61], the warning that it can underestimate by an exponential factor under any row or column ordering while appearing to have converged [62], and, closest in spirit to what Fish permits, polynomial approximate counting of tables with a constant number of rows [63]. Constrained deal generation in practice is done by rejection or by learning: GIB deals against suit-length restrictions and then discards deals whose bids its own bidding module would not have made — rejection sampling against a policy, one to two seconds for fifty worlds [12, 16] — while Skat engines enumerate and reweight with a learned per-card location model or policy-based reach probabilities [24, 25]. The general problem is hard: constructing a history consistent with a public state is FNP-complete in the worst case, and enumeration is polynomial only for *sparse* public trees [26].

Fish is unusually favourable. Cards move only by publicly observed transfer, so the hidden state never grows — it is the initial deal and nothing else — and each player's share of the initial deal stays constant at nine all game. The counting matrix therefore has at most six distinct column types, one per seat, and its permanent becomes polynomial rather than #P-hard: a dynamic program over the capacity vector costs about 5×10^5 multiply–adds from a player's seat. Two caveats deserve recording: tractability needs the constant column count *and* capacities that are small numbers written in unary, since counting contingency tables is #P-complete even with two rows once margins are binary-encoded; and ask legality is disjunctive, which would normally destroy the product form — the escape being that every legality certificate is confined to a single half-suit, so exhaustive enumeration within a block keeps it exact. A reference implementation with nine blocks and 27 exact legality constraints runs in 0.72 seconds of unoptimised NumPy, milliseconds once ported to C. Nothing comparable holds in Bridge, where the constraints are soft auction inferences and no such collapse of column types occurs.

3.6 Evaluation methodology

Card-game results are dominated by deal variance, so the standard remedy is a matched design: duplicate bridge replays each board with the same cards in the same seats, and the computer poker competition adds common seeds across pairings, reportedly cutting the standard deviation to about two thirds — a figure the corpus could verify only partially [64]. The resampling unit is then the deal, not the game, since replays of one deal form one correlated cluster; and the raw percentile bootstrap should be avoided, having been measured to fail at 5.7 to 11.2% against a nominal 5% [77]. Poker also supplies control variates: advantage-sum estimators [65], importance sampling over imaginary observations [66], MIVAT's least-squares value function [67], and AIVAT, which cut standard deviation by 68.8% in heads-up no-limit hold'em and 99.9% in Leduc with full knowledge [68, 70]. The gains are domain-sensitive: MIVAT delivered only about eighteen to twenty percent in six-player poker, the closest analogue to our setting.

For absolute quality the tools are best-response computation and its relaxation, local best response, which showed every entrant in the 2016 computer poker competition to be exploitable for more than 3,180 milli-big-blinds per hand [71]. Two lessons carry over verbatim: it gives only a lower bound, so a low score proves nothing — it lost 536 against a bot whose true exploitability in the same action set was 90 — and the point at which the exploiter may act must be swept, since restricting it to later rounds raised measured exploitability by nearly an order of magnitude. Refining an abstraction likewise does not monotonically reduce exploitability [72]. For ranking a population, Bradley–Terry-style models [73, 74] fail twice over when that population is a training run's own checkpoints: they are not redundancy-invariant, so duplicating one agent redistributes everyone's rating, and they cannot represent cycles. Balduzzi and coauthors offer a maximum-entropy Nash average over the antisymmetric logit matrix instead [75, 76].

Three documented traps shaped our protocol. Monitoring an interval computed for a fixed sample size and stopping when it looks good is not a test: in simulation it produced a 61.35% false-positive rate under the null [70], which is why sequential designs on normalised Elo exist [78]. Resampling games rather than deals inflates apparent precision whenever several games share a deal. And fitting a variance-reduction heuristic on the evaluation data destroys the guarantee that motivated it: AIVAT is unbiased for *any* value function, which constrains its expectation and not the realised estimate, and [69] tuned the heuristic on a published ten-thousand-hand six-player dataset until all fourteen players simultaneously appeared to be large winners — which the zero-sum structure makes impossible. Accordingly §9 uses six-rotation duplicate blocks, resamples deals and not games, freezes its held-out seed banks before the configuration is fixed, and uses a fitted exploiter as its only exploitability proxy. We deliberately do not use AIVAT or MIVAT: a learned control variate fitted by us, on our own agent, is the exact pathology this literature warns about.

4 The simulator: rules-exactness, information safety, verification

4.1 Implementation

The v0.4 engine is a single-header C++20 core. A hand is one `uint64_t` over the 54 card indices $c = 6h + i$, so a half-suit mask is `0x3F << 6h` and the six hands plus the deal they came from occupy under a hundred bytes. Where the agent explores a hypothetical — the two-ply refinement of §7.3 — it copies the constraint state and recomputes, rather than making and unmaking moves; there are no rollouts anywhere in this agent. Games are driven by a loop that, after every public event, (i) offers every player the chance to declare, (ii) resolves a cardless team into the forced endgame, (iii) resolves a cardless turn-holder into a chosen successor, and (iv) otherwise asks the turn-holder for a move. All

randomness comes from counter-based streams keyed by *(seed, seat, purpose)*, so a deal is reproducible from its seed, which the duplicate-block protocol of §9 requires. The verification suite checks replay identity at a fixed thread count; the streams are keyed so that thread count cannot matter, but we do not test that claim directly.

Agents are handed their own hand once and a stream of public events thereafter. There is no interface through which a policy can read another player's cards; the event record carries only actor, target, named card, outcome, declared allocation, and the resulting public hand counts.

4.2 Information safety, and why v0.3 needed an audit

The v0.3 audit found that its predecessor's reply-risk feature estimated a target's follow-up options by invoking the legal-move generator with the target as actor — which consults that target's hidden hand [1]. v0.4 removes the possibility structurally rather than by inspection: the acting policy never receives a pointer to the game state, only to its own `Knowledge` object and the public record. To confirm that no leak remains and, more importantly, that the belief machinery is *sound*, the engine has an audit mode that runs after every event and checks, for every player:

- every card whose holder that player believes is known is in fact held by that player;
- for every unresolved card, the true holder is contained in the surviving support M_c;
- the derived capacities (1) equal the true counts of unresolved cards per player;
- every live ask-legality certificate (2) is in fact satisfied by the true deal.

A single violation would mean the belief engine had excluded the truth — an unsound deduction. Across the verification sweep the engine performed **23,594,580** such checks with **0** violations (§10.1).

4.3 Verification suite

Beyond the audit the suite checks deck integrity and card conservation, that the nine half-suits are always exactly resolved, that declarations only ever assign cards to the declarer's own teammates, that ask legality is enforced on both the generator and the applier, that replays are bit-identical under a fixed seed, and that the action cap is not reached in ordinary play. The suite runs the full 8×8 policy cross-product so that pathological policies are exercised too.

4.4 Termination

Nothing in the rules forces a player to declare, so two sufficiently patient teams can in principle stall forever — and by Theorem 2, patient play is exactly what a strong policy discovers. We handle this the way the rules would in practice, with an in-policy cashing rule (§6.3), and keep a hard cap of 400 asks purely as a backstop. If it is ever reached, the unresolved half-suits are split by who physically holds the majority, with ties going to the holder of the lowest card — a neutral fallback rather than a rule, and one whose incidence therefore has to travel with every number. In the head-to-head runs of §10 it is **0.000%**; across the rule-dialect runs it stays below 0.7% of deals; the one place it is material is the mirror match inside the exploitability probe, which §12 reports separately.

5 Exact public-belief inference

5.1 The hidden state is the initial deal

Every card in Fish changes hands in exactly one way: a successful ask, which every player observes. Declarations remove cards, and that too is public. Hence:

Proposition 1 (Public transfer)**.** *Let c be a card whose location has never been publicly revealed. Then c is still held by the player to whom it was dealt. Consequently the joint hidden state of the game at any time is exactly the initial deal, and the current holder of every card is a deterministic function of the initial deal and the public history.*

Proposition 1 is what makes Fish unusual among card games with hidden information: it is an exact *public belief state* game. Every player — including every opponent — computes the same posterior over deals from the same public history, differing only by the private conditioning on their own hand. There is no filtering recursion to approximate and no state that decays.

5.2 The constraint system

Fix an observer i. Let U be the set of cards in still-live half-suits whose holder is not publicly known to i, and for $c \in U$ let $\sigma(c) \in \{0, \dots, 5\}$ be its (constant, by Prop. 1) holder. The observer's information is exactly the following constraints.

(C1) Own hand.
$\sigma(c) \neq i$ for every $c \in U$, and every card i holds is resolved.

(C2) Public transfers and correct declarations.
Any card whose holder has been revealed leaves U with its holder known exactly. A successful ask reveals the holder *before* the transfer, which is the value relevant to every earlier constraint.

(C3) Exclusions.
An ask for c proves the asker did not hold c; a miss proves the target did not hold c. Both are permanent, again by Prop. 1. Write $M_c \subseteq \{0, \dots, 5\}$ for the surviving support of $\sigma(c)$.

(C4) Capacities.
Hand counts are public, so for every player p

$$\big|\{c \in U : \sigma(c) = p\}\big| \;=\; q_p \;:=\; |H_p| - k_p, \tag{1}$$

where $|H_p|$ is p's public card count and k_p the number of live cards already known to be p's. Note that $\sum_p q_p = |U|$ automatically, and that a declaration — correct or not — is absorbed by (1) without special handling, because the count drop it causes is public.

(C5) Ask legality.
An ask for c in half-suit S by player A proves that A held some *other* card of S at that moment. Because unresolved cards never move, this is the time-independent disjunction

$$\bigvee_{d \in D} [\sigma(d) = A], \qquad D = \{d \in S \setminus \{c\} : d \text{ unresolved}, A \in M_d\}, \tag{2}$$

vacuous if some card of S is already known to be A's. Every such certificate is confined to a single half-suit.

The posterior is uniform over the assignments σ satisfying (C1)–(C5). This is the *grounded* or policy-agnostic posterior: it conditions on everything the rules make deducible and on nothing about how opponents choose their moves. §5.6 takes up the residual.

5.3 Counting: a capacity-vector dynamic program

Ignoring (C5) for a moment, counting assignments subject to (C3) and (C4) is a degree-constrained bipartite counting problem — the permanent of a 0/1 matrix with at most six distinct column types. Order U as $c_1, \dots, c_{|U|}$ and let the dynamic-programming state $n = (n_0, \dots, n_5)$ be the vector of *remaining* capacities, so that q is the full vector of (1). Write $F_j(n)$ for the number of ways to place the first j cards leaving n unused, and $B_j(n)$ for the number of ways to place cards $c_{j+1}, \dots, c_{|U|}$ out of n:

$$F_j(n) = \sum_{p \in M_{c_j} : n_p < q_p} F_{j-1}(n + e_p), \qquad F_0(q) = 1, \tag{3}$$

$$B_j(n) = \sum_{p \in M_{c_{j+1}} : n_p \geq 1} B_{j+1}(n - e_p), \qquad B_{|U|}(\mathbf{0}) = 1, \tag{4}$$

$$Z = F_{|U|}(\mathbf{0}) \;=\; B_0(q). \tag{5}$$

The per-card marginals are then obtained by splitting every consistent deal at card j:

$$\mu_{c_j,p} \;=\; \Pr[\sigma(c_j) = p \mid h] \;=\; \frac{\mathbf{1}[\,p \in M_{c_j}\,]}{Z} \sum_{\substack{n\,:\,|n| = |U|-j+1 \\ n_p \geq 1}} F_{j-1}(n)\; B_j(n - e_p). \tag{6}$$

The implementation detail that makes this cheap is that $F_j(n)$ is nonzero only when $\sum_p n_p = |U| - j$, so the layer index is *determined by the state*: F and B are each a single array of size $\prod_p (q_p + 1)$, not one array per card. Since the observer's own capacity is zero and $\sum_p q_p = |U| \leq 45$ over the five remaining players, AM–GM gives

$$\prod_{p \neq i} (q_p + 1) \;\leq\; \left(\tfrac{|U|}{5} + 1\right)^5 \;\leq\; 10^5, \tag{7}$$

so the entire forward–backward pass costs $O(6 \prod_p (q_p + 1)) \leq 6 \times 10^5$ multiply–adds regardless of the position. Sampling a deal exactly and without rejection follows from the backward table, $\Pr[\sigma(c_j) = p \mid \text{prefix}] \propto B_j(n - e_p)$; every prefix is extendable by construction. This is rejection-free for (C1)–(C4); with a live certificate (C5) the same sampler is used with an accept/reject test, which is exact but no longer rejection-free, and is how the validation of §10.1 obtains ground truth.

5.4 Ask legality by half-suit block enumeration

Constraint (C5) is disjunctive and breaks the product form. Inclusion–exclusion over k certificates costs 2^k passes, and a real game accumulates tens of them, so that is not an option. The structural escape is that every certificate lives inside one half-suit. We therefore partition U by half-suit and give each block one of two exact treatments.

- A block with no live certificate is expanded card by card, exactly as in (3)–(4), with branching factor $|M_c| \leq 5$.
- A block with at least one live certificate is enumerated exhaustively — at most $\prod_{c \in S} |M_c| \leq 5^6$ assignments — each survivor is checked against (2) directly, and the survivors are aggregated into a count-vector table

$$g_S(t) \;=\; \big|\{\text{assignments } a \text{ of } S : \text{count}(a) = t,\; a \models \text{certificates of } S\}\big|, \qquad \textstyle\sum_p t_p = m_S. \tag{8}$$

Either way a block consumes a known number of cards, so the layer identity survives and the dynamic program simply runs over blocks: $F_b(n) = \sum_t g_b(t)\, F_{b-1}(n + t)$. Writing S_t for the path mass of count

vector t at its block's entry layer,

$$S_t \;=\; \sum_{|n|\,=\,\text{entry layer}} F(n)\,B(n-t), \tag{9}$$

the three queries the policy needs are all exact and all read straight off the tables:

$$\mu_{c,p} = \frac{1}{Z}\sum_t g_S^{(c\to p)}(t)\,S_t \qquad \text{(per-card ownership)} \tag{10}$$

$$\Pr[\text{allocation } A] = \frac{S_{t(A)}}{Z} \qquad \text{(the declaration query)} \tag{11}$$

$$\Pr[\text{team } T \text{ owns } S] = \frac{1}{Z}\sum_{t\,:\,\mathrm{supp}(t)\subseteq T} g_S(t)\,S_t \qquad \text{(is the half-suit ours?)} \tag{12}$$

Equation (11) deserves emphasis. Declaring requires naming an exact allocation, so the quantity that governs the decision is the joint probability that all six assignments are simultaneously right — *not* a product of per-card marginals, which is badly wrong because the six cards of a half-suit are strongly dependent through (1). FishBot v0.3 used a geometric mean of per-card marginals; v0.4 uses (11).

A corollary of (11) that we did not anticipate: under the uniform prior, every surviving allocation sharing a count vector has *identical* probability. The MAP allocation is therefore determined by the count vector alone, and the choice among allocations within a count vector is arbitrary.

5.5 A fast approximate posterior, and what it costs

The exact engine is affordable per query but is invoked by six observers after every public event, which puts it in the inner loop of any large-scale experiment. We therefore also implement an approximation that keeps the exact constraint bookkeeping (C1)–(C3) and the exact capacities (C4), fits them by Sinkhorn scaling, and interleaves a conditioning step for (C5). For a certificate $\bigvee_{d\in D}[\sigma(d)=A]$ with event E, independence conditioning gives

$$\Pr[\sigma(d)=A\mid E] = \frac{p_{d,A}}{1-\prod_{e\in D}(1-p_{e,A})}, \qquad \Pr[\sigma(d)=p\mid E] = p_{d,p}\,\frac{1-\prod_{e\in D\setminus\{d\}}(1-p_{e,A})}{1-\prod_{e\in D}(1-p_{e,A})}, \tag{13}$$

after which capacity fitting is re-run; four outer sweeps of eight Sinkhorn iterations converge in practice. The cost is $O(\text{iters}\cdot|U|\cdot 6)$, roughly two orders of magnitude below the exact program.

§10.1 reports the measured gap: the approximation is accurate on average but has a heavy tail, and — more interestingly — the strength difference between exact and approximate inference depends on which of the two the ask policy was fitted against.

5.6 The residual: policy-aware inference

The posterior of (C1)–(C5) is exact with respect to the rules, but it is not the full Bayesian posterior. That would additionally weight each deal x by $L(h\mid x)=\prod_t \pi_{p_t}(a_t\mid h_{<t},x)$, the likelihood that the observed actions would have been chosen given x — which depends on the opponents' policies and is therefore *playstyle dependent*. Concretely, a player who has taken many turns and never asked in half-suit S is, under any reasonable policy, less likely to hold cards of S; the rules alone say nothing about this.

We implement the natural one-parameter family of such corrections as prior weights inside the same machinery,

$$w(c,p) \;=\; \exp\!\big(\theta\,a_{p,S(c)} - \phi\,(a_p - a_{p,S(c)})\big), \tag{14}$$

where $a_{p,S}$ counts p's public asks in S and a_p their total asks, and we fit θ, ϕ jointly with the rest of the policy. Setting $\theta = \phi = 0$ recovers the grounded posterior. Equation (14) with $\phi = 0$ is exactly the heuristic v0.3 used in place of the certificate (2). The measured outcome (§11) is that once the hard certificate is enforced, the soft weight adds nothing — evidence, though not proof, that (2) already captures the bulk of the signal that v0.3 was approximating.

6 When to declare: a structural result and an optimal-stopping rule

6.1 A half-suit held by one team cannot be taken back

Every prior Fish agent we are aware of, including FishBot v0.3, declares as soon as its confidence clears a threshold. A threshold is a defensible rule — our own ablations cannot separate it from the alternative — but it is answering the wrong question, because under the real rules the risk it is guarding against does not always exist. The reason is a one-line consequence of ask legality.

Theorem 2 (Locked half-suits are safe)**.** *Suppose every card of half-suit S is held by members of team T. Then no member of the opposing team can ever legally ask for a card of S, so no card of S can change hands, and S remains wholly held by T for as long as it stays undeclared. Moreover any declaration of S by the opposing team is necessarily incorrect and awards S to T.*

Proof. A legal ask for a card of S requires the asker to hold another card of S. Since all six cards of S are held by members of T, every opposing player is void in S and no such ask is available to them — and a player holding no cards at all has no legal ask of any kind. Cards move only through successful asks, so no card of S can leave T; the only asks in S available to anyone are asks by members of T directed at opponents, which are guaranteed misses and transfer nothing. Ownership of S is therefore frozen. For the second part, a declaration must assign every card of S to a teammate of the declarer; an opposing declarer would be assigning cards held by T to members of the other team, which is false, so the declaration fails and S is awarded to T. □

The theorem deliberately does not assert that the probability of a correct claim rises monotonically. It does not: with $\Pr_t[A] = N_t(A)/Z_t$ and both counts shrinking as constraints accumulate, a deduction elsewhere in the deal can eliminate more completions of the true allocation than of a rival and drive the ratio down. What *is* monotone is the support — the set of allocations of S extendable to a deal consistent with the public history is non-increasing — and that is all the argument below needs.

Corollary 3 (Waiting is risk-free)**.** *For a locked half-suit the "steal" term in the declaration trade-off is identically zero: the only ways S can leave the undeclared state are a declaration by T, or a declaration by the opposing team which by Theorem 2 awards S to T anyway. Declaring immediately can therefore never protect the half-suit; the only reasons to declare are the point itself and whatever positional effects the removal of six cards has.*

Theorem 2 is the reason declaration timing is a genuine decision rather than a threshold. Two effects push in opposite directions.

Waiting hides information. Declaring a half-suit resolves six cards that were, from an opponent's point of view, part of the unresolved pool. By (1) that tightens every opponent's capacity constraint on the cards that remain. Holding the cards keeps those six slots absorbing probability mass the opponents cannot allocate.

It is worth being precise about what a locked half-suit does and does not admit, because the tempting statement is wrong. Asks inside it transfer no card — the target is void — but they are still *legal* for the owning team and they still carry the certificate (2), which is information about which teammate holds what. Develin notes the same thing from the human side: asks inside a half-suit your team already owns are a channel the opponents cannot use. So the allocation of a locked half-suit can be sharpened deliberately, at the price of a turn; it also sharpens for free as other cards resolve and the capacities tighten. FishBot v0.4 does not exploit the deliberate version, and §14 lists it as the clearest unexploited idea we know of.

Waiting wastes turns. A card of a locked half-suit is nearly dead weight: it licenses only asks that are certain to fail. A player whose hand is mostly locked cards has a turn worth little, whereas a player who declares and empties their hand exits the asking cycle and hands the turn to a teammate who can use it. Cards also invite being asked.

6.2 Declaration as optimal stopping

We therefore treat declaration as a stopping problem against a value function rather than a confidence threshold. Let $V(\cdot)$ be the static evaluation of §7.4, in units of final set differential scaled to $[-1, 1]$, and let $q_t = \Pr[\text{the MAP allocation of } S \text{ is exactly right}]$ from (11). Declaring replaces the half-suit's contribution to the position by a scored point, correct with probability q_t:

$$\text{declare } S \iff q_t\, V(s \ominus S,\, \Delta{+}1) + (1 - q_t)\, V(s \ominus S,\, \Delta{-}1) \;>\; V(s) + \varepsilon, \tag{15}$$

where $s \ominus S$ is the position with S removed — six fewer cards in the team's hands, six fewer unresolved slots, one fewer live half-suit — and ε is a fitted margin. The immediate expected score change is $2q_t - 1$; but because a fitted V already credits expected control of S, the $2q_t - 1$ term largely cancels and (15) is decided by exactly the positional terms Corollary 3 says are the real considerations. When $q_t = 1$ and S is locked, (15) reduces to a comparison between the information value of the six absorbing slots and the turn efficiency of a lighter hand.

The behaviour this produces is measurable, and it does not go the way we first guessed. In the held-out head-to-head, the fitted v0.4 sits on a half-suit that has become provably locked for 5.2 public events before cashing it, while FishBot v0.3 sits on one for 14.8 — v0.4 is the *less* patient of the two. (Both figures are purely observational: the moment a half-suit becomes locked is computed from ground truth after the fact and is never available to either policy. Both are also conditioned on the declaration being correct, and the two policies declare correctly at very different rates, so the two averages are taken over differently selected subsets — the direction of the difference is safe, its exact size is not.)

The explanation is that the two policies wait for different reasons. FishBot v0.3 waits because it cannot resolve the allocation: its confidence is a geometric mean of per-card marginals and it needs many more public events before that clears its threshold. FishBot v0.4 knows the exact allocation probability (11) much sooner, so once the stopping rule is satisfied it cashes. Theorem 2 says waiting is *risk-free*; it does not say waiting is profitable, and the fitted policy's answer is that the information the six absorbing slots hide is not worth the turn efficiency they cost. That is an empirical answer to an open question, not a theorem, and it is specific to this opponent population.

6.3 Termination is a property of the policy, not of the rules

Corollary 3 has a consequence that only shows up when both sides act on it. Nothing in the rules of Fish compels anyone to declare. Consider a position in which every live half-suit is wholly owned by one team

or the other, but at least one team cannot resolve which of its members holds which card. By Theorem 2 no card can move: every opponent is void in every live half-suit, so every legal ask is a guaranteed miss. The public state is therefore frozen, no further information can ever arrive, and the allocation that was uncertain stays uncertain. Two policies that both correctly refuse to cash an uncertain half-suit will sit in that position forever.

This is not hypothetical. An earlier fitted configuration, played against a copy of itself, failed to terminate in 21% of deals; raising the action cap from 400 asks to 1000 left the figure at 21%, which distinguishes a genuine cycle from slow convergence. Against differently-shaped opponents the same configuration terminated far more reliably, because an opponent that is not a near-copy breaks the symmetry; `research/v04/results/E13-termination.jsonl` reports the rate against every member of the population.

The fix has to come from the policy, and the obvious version of it is wrong. The natural test — "declare when no productive ask remains" — also fires in ordinary early positions, where a player who happens to hold only cards their own team already owns still has every reason to wait; wiring it in cost 28 win-rate points against FishBot v0.3. What works is a graduated escalation on the public event count: at ordinary pressure the stopping rule (15) decides; past a forcing horizon the policy cashes any half-suit it is better than even money on; past a second horizon it cashes its best candidate whatever the probability, on the grounds that an unclaimed half-suit scores nothing at all. With this rule every game in every experiment in this paper terminates through play, and the action cap is never reached.

We take the underlying observation to be a fact about the rules rather than about our agent: the 54-card Fish rules as usually stated do not guarantee termination against deterministic play, and a tournament form of the game would want an explicit forced-claims provision. The suggestion is not new — the rules literature records such a proposal [2] — but the reason it is necessary is, as far as we can tell, unstated.

7 FishBot v0.4

7.1 Overview

FishBot v0.4 is a deterministic, fully inspectable policy with four parts: an exact constraint tracker, a posterior over deals (§5), a scored choice over legal asks, and a decision-theoretic declaration module (§6). It sees only its own hand and the public record. There is no neural network, no tree search, and no private channel between teammates; everything a teammate learns, an opponent learns at the same instant.

7.2 Choosing an ask

Every legal pair $a = (c, t)$ of named card and opposing target is scored as

$$U(a) \;=\; \lambda_{\text{lin}} \sum_{k=0}^{19} w_k \, \varphi_k(a) \;+\; \lambda_{\text{val}} \Big[p\, V(s_a^{+}) + (1-p)\, V(s_a^{-}) \Big], \tag{16}$$

where $p = \mu_{c,t}$ is the exact posterior that t holds c, φ is the feature vector of Table 1, V is the value function of §7.4, and $s_a^{\pm}$ are the positions after a hit and after a miss. The bracketed term is a one-ply expectimax: a hit moves the card into our hand and retains the turn, while a miss excludes the target — which renormalises that card's ownership over the remaining candidates, so a miss is informative as well as costly — and hands the turn over. Ties break deterministically by card then target index, and the chosen ask's p is retained as a forecast for the calibration study.

Table 1: Ask features. All are normalised to roughly $[0, 1]$ so that fitted weights are comparable. S is the named card's half-suit.

k	Name	Definition
0	hit probability	$p = \mu_{c,t}$, exact posterior that the target holds the card
1	squared hit	p^2; lets the fit curve the trade-off between certainty and value
2	certain hit	$\mathbf{1}[p > 0.9995]$
3	own progress	cards of S in hand, over six
4	team control	expected fraction of S held by our team
5	lock completion	$\prod_{d \in S \setminus \{c\}} \Pr[d \text{ is ours}]$: the chance this ask alone locks the half-suit
6	continuation	best posterior on any other missing card of S
7	completion bonus	1 at four or more cards of S in hand, 0.35 at three
8	reply threat	$(1 - p)\cdot$ the best ask the target could make against us if handed the turn
9	information leak	1 if our team has never publicly revealed an interest in S
10	target hand size	target's public card count over nine
11	empties the target	p if the target holds exactly one card, else 0
12	repeats our suit	1 if this repeats our own previous half-suit
13	known team cards	publicly located cards of S on our team, over six
14	location entropy	$H_2(p)$
15	team owns S	$\prod_{d \in S} \Pr[d \text{ is ours}]$ before the ask; an independence proxy, not the exact (12)
16	exposure on a miss	$(1 - p)\cdot$ our cards visible in half-suits the target has asked in
17	trailing pressure	p when behind by two or more half-suits
18	runway	expected length of the run this ask begins, from the best remaining per-card posteriors
19	leak magnitude	feature 9 scaled by how much of S we hold

Feature 18 deserves a note. A hit retains the turn, so the value of an ask is not one card but the expected length of the run it starts. Because a legal ask already requires a card of that half-suit in hand, a hit cannot open a half-suit that was previously unavailable; the continuation is therefore well approximated by the best remaining per-card posteriors, nested as $p\,(1 + q_1(1 + q_2(1 + q_3)))$ with $q_1 \geq q_2 \geq q_3$ the next-best probabilities.

7.3 An exact two-ply refinement

Equation (16) evaluates both branches of an ask against the *current* posterior, and features 6 and 18 approximate what happens after it. For the leading candidates we can afford to stop approximating. We rank all legal asks by (16), keep the top K, and for each of those re-derive the posterior on both branches — a hit resolves the card to our hand and adjusts two card counts; a miss adds an exclusion — and then measure

$$\text{follow}(a) = \max_{a'} \mu'_{a'} \quad \text{over our legal asks in the post-hit position;} \tag{17}$$

$$\text{threat}(a) = \max_{H} \Big(\Pr'\big[t \text{ holds a card of } H\big] \cdot \max_{c \in H} \Pr'\big[c \text{ is ours}\big] \Big) \quad \text{in the post-miss position,} \tag{18}$$

scoring the candidate as $U(a) + \lambda_{\text{chain}}\, p\, \text{follow}(a) - \lambda_{\text{threat}}\,(1 - p)\,\text{threat}(a)$. Both λ and K are fitted. Note what this is *not*: it is not a determinized search. Sampling a deal and searching it would be worse than useless here, because a clairvoyant player in Fish never fails an ask and never mis-declares, so the perfect-information game is degenerate and averaging over determinizations collapses toward "ask for the card most likely to be there" (§3). The refinement above stays inside the belief.

7.4 The value function

V maps a public belief state to an estimate of the final half-suit differential, scaled to $[-1, 1]$ and signed from the evaluating team's point of view. It is linear in sixteen features: a bias; the current score differential; the summed expected control $\sum_H (2e_H - 1)$ and a sharpened version of it, where e_H is the expected fraction of half-suit H our team holds; the signed count of half-suits locked for us minus those locked against us; side to move; the card differential between the teams; the size of the unresolved pool; the number of live half-suits; the interaction of side-to-move with control; our own hand size; our smallest friendly hand; the counts of contested half-suits leaning each way; total contested mass $\sum_H e_H(1 - e_H)$; and the interaction of side-to-move with the unresolved pool.

V is fitted by ridge regression on self-play, labelling every decision point with the eventual differential. Features are collected from *all six seats* at each decision, not only the mover: the side-to-move feature is otherwise constant at $+1$ and its coefficient — which is precisely what the miss branch of (16) and the stopping rule (15) depend on — would be unidentified.

7.5 Declaring

At every public event, each agent is offered the chance to declare. Running the exact posterior six times per event is far too costly, so a capacity-only gate runs first: for each live half-suit we form $\prod_{c \in S} \Pr[c \text{ is ours}]$ from capacities and supports alone, with no dynamic program, and skip half-suits that cannot plausibly be ours. Half-suits that survive the gate are then scored. Which estimator is used depends on the belief mode, and it matters that the shipping configuration uses the fast one: under `belief=block` the exact queries (11) and (12) are read straight off the block tables, but the shipped configuration computes Pr[allocation] by sequential conditioning — fix one card, decrement that owner's capacity, refit, multiply — which is exact in structure and approximate only in the marginal solver, and it names the allocation by taking the most likely teammate for each card under the unconditioned marginals. The exact engine's role is to establish how good those approximations are (§10.1) and to serve as the ablation they are measured against, not to run in the shipped inner loop.

Four fitted quantities decide when the stopping rule is overridden rather than consulted. The policy declares itself *pressed* when the unresolved pool has fallen below `patiencePool` cards, or the opposing team holds fewer than `oppCardFloor` cards between them, or its own best available ask has posterior below `askFloor`, or the public event count has passed the forcing horizon. When pressed, a plain probability threshold replaces (15). The third of these is a softened form of the dead-ask trigger that §6.3 reports as harmful in its hard form: as a *fitted* floor combined with a threshold rather than an unconditional force, the optimiser kept it, and it sits at a value where it fires rarely.

Above all of this sits the escalation of §6.3, which exists because Theorem 2 makes it possible for two patient policies to deadlock permanently. Below a forcing horizon on the public event count, (15) has the last word; above it, the policy cashes any half-suit it is better than even money on; above a second horizon it cashes its best candidate regardless, and the gates are relaxed to match so that a half-suit the policy is unsure of can still be named.

7.6 Turn gifts and the forced endgame

When a declaration leaves a player cardless and the turn reaches them, they hand it to the teammate with the strongest publicly-estimable ask: for each live teammate u we score $\max_H \Pr[u \text{ holds a card of } H] \cdot \max_{c \in H} \Pr[c \text{ is with an opponent}]$, which uses only public information and our own hand.

When a whole team is cardless, the other team must declare everything with no further asking, and

may exchange only willingness. We implement exactly that: the team sweeps a descending ladder of confidence thresholds $(0.995, 0.98, 0.95, 0.90, 0.80, 0.65, 0.50)$ and, at each level, any willing player declares any half-suit they are willing to declare. This recovers most of "the best-informed player goes first" while exchanging nothing but a bit per half-suit, and it matters because each correct declaration announces an exact allocation, which sharpens the capacity constraints for the declarations that follow.

8 Fitting

8.1 A population-robust objective

The question this project set out to answer is not "which policy has the best average score against a fixed pool" but "is there a single policy that is best across playstyles". A mean win rate answers the first and hides the second, so we fit against a soft minimum over the opponent panel $\mathcal{O}$:

$$\text{score}(w) \;=\; -\frac{1}{\beta}\,\log \sum_{o\in\mathcal{O}} \exp\!\big(-\beta\,\text{wr}_o(w)\big), \qquad \beta = 8, \tag{19}$$

which tends to $\min_o \text{wr}_o$ as β grows and is smooth enough for a noisy optimiser. The panel $\mathcal{O}$ holds six policies: FishBot v0.3, the turn-starvation lockout, the posterior detective, FishBot v0.2, the adaptive diversifier and the focused hunter. The misdirection artist and the random control are deliberately *excluded* so that they remain held-out opponents, as are all rule variants.

8.2 Optimiser

We use the cross-entropy method with a diagonal Gaussian: a population of candidates is sampled around the incumbent mean, the elite fraction is retained, and the mean and per-coordinate standard deviation are smoothed toward the elite statistics. CEM was chosen over gradient-free alternatives because the objective is a noisy win rate — exactly the regime in which methods that trust small differences fail. Two devices keep the noise manageable. First, every candidate in a generation is evaluated on *identical* seed banks, so comparisons within a generation are paired. Second, the banks are redrawn between generations, so the fit cannot lock onto a particular set of deals.

The parameter vector is fitted jointly and has 34 coordinates: the twenty ask weights of Table 1, the declaration threshold and the locked threshold, the ask floor, the patience pool size, the opponent-card floor, the value and linear mixing weights of (16), the minimum team probability, the stopping margin ε, the two policy-prior coefficients θ, ϕ of (14), and the three two-ply parameters of §7.3. Bounds are imposed per coordinate so that probabilities stay probabilities.

8.3 Schedule

Round one fitted the twenty ask weights alone with the fast posterior, on a four-opponent panel. Round two started from round one's mean and fitted all 31 coordinates on the six-opponent panel. The value function was fitted separately by ridge regression on self-play decision points, as described in §7.4, and then held fixed while the policy weights were fitted around it. Fitting used base seeds 20260821, 770077 and 313131. The shipping configuration was then chosen on a further validation bank (1357911) that the search itself never saw, because the per-generation best reported by the optimiser is a maximum over a population evaluated on shared seeds and is therefore upward biased; generation 8 won that selection. Every seed bank used in §10 onwards is disjoint from all of these, and the configuration was frozen before any of them was run.

Table 2 lists the frozen values.

Table 2: The frozen FishBot v0.4 configuration. Note that the locked-half-suit threshold and the patience switch are inert while the stopping rule is active (§11), so no meaning should be read into their fitted values.

Ask feature	Weight	Decision knob	Value
hit probability	+11.506	declare threshold	0.818
squared hit	+3.295	locked threshold	0.797
certain hit	+3.198	ask floor	0.333
set progress	+1.688	patience pool	6.249
team control	+2.133	opp. card floor	2.868
lock completion	+4.071	value weight	6.043
continuation	+1.468	linear weight	0.767
completion	+1.428	min. team prob.	0.792
reply threat	-3.098	stopping margin	-0.034
information leak	-0.854	prior θ	0.264
target hand size	-2.022	prior ϕ	0.133
empties target	+1.166	two-ply K	5.624
repeats set	+1.270	chain weight	3.358
known team cards	+0.914	threat weight	2.613
entropy	-2.653		
team owns set	-0.804		
exposure on miss	+1.904		
trailing pressure	+0.047		
runway	+1.411		
leak magnitude	-0.999		

9 Evaluation protocol

Fish has a single chance node — the deal — followed by a fully public transcript. That is close to the best case for variance reduction, and we exploit it.

9.1 Six-rotation duplicate blocks

Because seats alternate teams, the cyclic group of six seat rotations of a fixed deal forms a balanced block: the three odd rotations exchange which team holds which hand-triple, and the even rotations permute hands within a team. Playing all six rotations of every deal means each policy holds every hand-triple exactly once, so the intrinsic luck of the deal cancels out of the comparison rather than being averaged down. This is strictly stronger than the two-orientation swap used for v0.3, which balances only the team label.

9.2 Clustered inference

The six rotations of one deal are one correlated cluster, so resampling *games* would understate the variance. All bootstrap intervals in this paper resample *deals*: for a single arm we report a cluster bootstrap of the per-deal win count, and for a comparison between two configurations played on the same deals we report a paired bootstrap of the per-deal difference. Wilson intervals are also given for continuity with the v0.3 paper; where the two disagree, the bootstrap is the one to trust.

9.3 Seed separation

Fitting used base seeds 20260821, 770077, 313131, 888111 and 606111 across five rounds, and the shipping configuration was selected on a further validation bank (1357911). Every number reported in §10 onwards comes from banks disjoint from all six — 90210 for the head-to-head, 515151 for the matrix, 606060 for the ablations, 717171 for calibration, 828282–848484 for the rule dialects, 20260820 for the v0.3 port validation, and 515253/6543210 for the exploitability probe — and the configuration was frozen before any of them were run.

9.4 Metrics

Win rate is primary but insufficient on its own, for the reason §10 makes concrete: a policy can win by declaring better while asking worse. We therefore report, for every matchup:

Mean half-suits.
Out of nine; the margin, not just the sign.

Ask accuracy.
Fraction of asks that transfer a card.

Declaration accuracy and rate.
Voluntary declarations per game and the fraction scored correctly. Forced endgame declarations are counted separately in the win-rate tables; the calibration study pools the two, which is a negligible contamination at well under a tenth of a forced declaration per game.

Out-of-turn declarations.
Per game; zero under the v0.3 rule dialect.

Forecast calibration.
Brier score, log loss, expected calibration error and a ten-bin reliability table for the policy's own Pr[hit] and Pr[allocation correct] predictions. A policy whose declaration probabilities are miscalibrated is making its stopping decision on a corrupted scale, so this is a first-class result rather than a diagnostic.

Worst-case profile.
The minimum win rate across the opponent panel, not only the mean. A policy that averages well but collapses against one playstyle has not answered the question this project asks.

Local best response.
An explicit exploiter, fitted *against* a frozen configuration inside the same policy class, gives an upper bound estimate on how much a well-informed adversary who has studied the bot can win. This is the closest available proxy for exploitability in a game where exact best-response computation is out of reach.

10 Results

10.1 Engine and belief validation

The verification sweep plays every ordered pair of the eight policies with the knowledge audit of §4 active. Across 23,594,580 soundness checks it recorded 0 violations: no agent ever excluded the true holder of a card, no derived capacity ever disagreed with the truth, and no ask-legality certificate was ever violated by the actual deal. Card conservation and exact nine-half-suit resolution hold in every game, and replays are bit-identical under a fixed seed at a fixed thread count. The action cap fires on a handful of the verification sweep's deals — that sweep deliberately includes pathological policy pairings — and on none of the head-to-head deals reported below.

The belief engines were then checked against each other and against ground truth. With no ask-legality certificate live, the block-enumeration posterior and the card-level capacity dynamic program agree to 3.414×10^{-15} — machine precision, as they must, since they compute the same quantity two different ways. With certificates live, the block posterior agrees with exact rejection sampling from the constraint-satisfying distribution to 4.066×10^{-2}, which is Monte Carlo error at the sample sizes used. The fast approximation of §5.5 has mean absolute marginal error 0.01705 but a maximum of 0.4982: accurate on average, with a heavy tail exactly where the position is sharp.

10.2 Validating the v0.3 port

Every comparison in this paper is against a re-implementation of FishBot v0.3 in the new engine, so that re-implementation has to be shown faithful. Run under the v0.3 rule dialect on the v0.3 seed bank, the port reproduces six of the seven published figures within their confidence intervals (Table 3); the seventh, against the misdirection artist, misses by half a point at the top of the scale, where both figures are above 98%.

Table 3: FishBot v0.3 as published versus the C++ port, under the v0.3 rule dialect. Published values are from [1].

Opponent	Published v0.3	C++ port	95% CI
Turn-starvation lockout	57.85%	57.60%	55.5–59.8
Posterior detective	57.20%	59.10%	57.0–61.3
FishBot v0.2	56.50%	56.60%	54.5–58.7
Adaptive diversifier	86.15%	84.75%	83.2–86.3
Focused hunter	95.15%	94.55%	93.5–95.5
Misdirection artist	99.20%	98.70%	98.2–99.2
Random legal control	100.00%	100.00%	100.0–100.0

10.3 Held-out head-to-head

Table 4 gives the primary result: FishBot v0.4 against the whole population on an untouched seed bank, with every deal played in all six seat rotations.

Table 4: FishBot v0.4 on the held-out bank, six-rotation duplicate blocks, 4,200 games per row.

Opponent	Win rate	95% CI	Mean sets	Ask acc.	Decl. acc.	Decl./game
FishBot v0.3	75.07%	73.7–76.4	5.457	55.81%	98.55%	4.80
Turn-starvation lockout	78.12%	76.9–79.4	5.542	48.01%	98.14%	5.16
Posterior detective	75.74%	74.5–77.0	5.440	52.79%	98.36%	5.12
FishBot v0.2	84.24%	83.1–85.3	5.857	54.42%	98.85%	5.17
Adaptive diversifier	92.14%	91.3–93.0	6.442	64.05%	97.42%	6.14
Focused hunter	97.64%	97.2–98.1	7.000	55.69%	97.51%	5.09
Misdirection artist	99.95%	99.9–100.0	8.161	55.12%	98.16%	3.80
Random legal control	100.00%	100.0–100.0	8.579	54.26%	96.72%	6.99

The number that matters for the question this project asks is not the mean but the minimum: FishBot v0.4's *worst* result against any policy in the population is 75.07%. Against the strongest predecessor it wins 75.07% (95% CI 73.71–76.40), taking 5.457 of the nine half-suits on average. For scale, FishBot v0.3's own worst result against the three credible challengers of its study was 56.50% [1].

Ask accuracy does not explain any of this, and the clearest evidence is the row where it goes backwards. Against the turn-starvation lockout policy v0.4 converts a far *lower* fraction of its asks than against any other opponent, and yet beats it by a wider margin than it beats either of the other two policies of comparable strength. Two things are happening. The lockout policy is built to refuse the turn to a dangerous opponent, which means it hands v0.4 positions in which the remaining asks are genuinely poor; and v0.4's edge is concentrated not in converting asks but in declaring — it claims at roughly 98.55% accuracy while FishBot v0.3, on those same games, claims correctly only 82.02% of the time, and every failed claim is a half-suit handed to the opponents. A policy can ask worse and still win, provided it never gives a set away.

The rule permitting a claim at any moment is used heavily: v0.4 makes 3.43 out-of-turn declarations per game against v0.3. §12 isolates how much of the margin depends on it.

10.4 Population ratings

Table 5 converts the round-robin of Table 6 into Bradley–Terry ratings.

Table 5: Bradley–Terry ratings over the round-robin, expressed in Elo with FishBot v0.3 at zero.

Policy	Elo (v0.3 = 0)	Mean win rate
FishBot v0.4	+216	87.73%
Posterior detective	+8	68.84%
Turn-starvation lockout	+5	68.48%
FishBot v0.3	+0	68.02%
FishBot v0.2	-32	64.74%
Adaptive diversifier	-218	46.46%
Focused hunter	-441	29.60%
Random legal control	-759	13.26%
Misdirection artist	-1029	2.86%

Table 6: Round-robin win rates (%), row policy against column policy, six rotations per deal.

Row vs column	v0.4	v0.3	v0.2	Lock	Infer	Divers	Focus	Bluff	Rand
FishBot v0.4	—	75.7	83.5	78.7	74.0	92.7	97.3	99.9	100.0
FishBot v0.3	24.3	—	51.5	49.5	49.8	78.8	91.0	99.3	99.9
FishBot v0.2	16.5	48.5	—	41.6	44.8	76.0	91.0	99.6	99.9
Turn-starvation lockout	21.3	50.5	58.4	—	49.9	77.0	91.2	99.5	100.0
Posterior detective	26.0	50.2	55.2	50.1	—	76.9	92.8	99.5	100.0
Adaptive diversifier	7.3	21.2	24.0	23.0	23.1	—	74.4	99.5	99.3
Focused hunter	2.7	9.0	9.0	8.8	7.2	25.6	—	93.1	81.5
Misdirection artist	0.1	0.7	0.4	0.5	0.5	0.5	6.9	—	13.4
Random legal control	0.0	0.1	0.1	0.0	0.0	0.7	18.5	86.6	—

The shape of Table 5 is worth a sentence. FishBot v0.3, the turn-starvation lockout and the posterior detective are separated by four Elo points — for practical purposes they are the same strength, which is what one would expect of three policies that all reduce to "ask where the posterior is highest, and be careful about claiming". FishBot v0.4 sits +216 Elo clear of that group. The gap is not a refinement of the same idea; it comes from replacing the posterior itself and from what the exact posterior makes possible at the moment of the claim.

10.5 Throughput

The fitted configuration plays roughly twenty times faster than the same policy driven by the exact block posterior (`research/v04/results/E9-throughput.txt`), which is why the exact engine is used to validate probabilities and to serve as an ablation rather than to run in the inner loop. Both are far slower than FishBot v0.3, which does no exact inference at all; the absolute rates are fast enough that every experiment in this paper is a matter of minutes.

10.6 Calibration

A stopping rule is only as good as the probabilities it stops on, so the policy's own forecasts are a first-class result rather than a diagnostic. Table 7 gives the aggregate scores and Table 8 the decile breakdown.

Table 7: Reliability of the policies' own probability forecasts on held-out games. FishBot v0.3 does not expose an ask forecast, so only its declaration confidence is comparable.

Policy	Forecast	n	Brier	Log loss	ECE	Mean pred / obs
FishBot v0.4	Pr[ask succeeds]	56,204	0.1297	0.3790	0.0288	0.5338 / 0.5504
FishBot v0.4	Pr[allocation correct]	5,816	0.0155	0.0592	0.0222	0.9575 / 0.9789
FishBot v0.3	Pr[allocation correct]	4,984	0.1339	0.3990	0.1257	0.9459 / 0.8202

Table 8: Reliability of both forecasts, by decile of the predicted probability. The declaration column is where (15) operates, and it is sparse everywhere except the top bin.

	Pr[ask succeeds]			Pr[allocation correct]		
Forecast bin	n	forecast	observed	n	forecast	observed
$[0.0, 0.1)$	6,341	0.0291	0.0126	34	0.0000	0.0000
$[0.1, 0.2)$	4,711	0.1557	0.1407	—	—	—
$[0.2, 0.3)$	7,023	0.2449	0.2876	—	—	—
$[0.3, 0.4)$	6,547	0.3483	0.4100	—	—	—
$[0.4, 0.5)$	5,096	0.4496	0.5057	—	—	—
$[0.5, 0.6)$	4,681	0.5402	0.5791	88	0.5339	0.5227
$[0.6, 0.7)$	2,216	0.6446	0.6918	74	0.6624	0.8243
$[0.7, 0.8)$	2,089	0.7522	0.7037	415	0.7434	0.9663
$[0.8, 0.9)$	1,359	0.8496	0.8205	221	0.8413	0.9412
$[0.9, 1.0)$	16,141	0.9980	0.9963	4,984	0.9989	0.9986

Declaration forecasts are calibrated to within 0.0222 expected calibration error, with realised accuracy 97.89%.

That aggregate needs a qualification, because it is dominated by one bin. Roughly nine tenths of the agent's declaration forecasts sit above 0.99, where it is almost perfectly calibrated; the sparse middle bins, between 0.6 and 0.9, are underconfident by eight to nineteen points. Those are exactly the forecasts at which (15) has a decision to make rather than a formality to discharge. The direction is the safe one — the agent claims less often than it should, not more — and it is consistent with the ablation finding that moving the declaration threshold between 0.80 and 0.99 changes nothing measurable, since so little mass lies between them. But the honest statement is that the calibration result licenses the *scale* the stopping rule operates on, not its behaviour in the region where it is actually deciding.

The comparison with the predecessor is the sharpest single diagnostic in the study, and it explains the

declaration-accuracy gap in Table 4 mechanically rather than by appeal to a better policy. FishBot v0.3's declaration confidence is a geometric mean of per-card marginals; on the same games it predicts 94.59% and achieves 82.02%, an expected calibration error of 0.1257 — roughly eight times v0.4's, and in the dangerous direction. A policy that believes it is 95% sure when it is 84% sure will keep making claims it should not make, whatever threshold sits on top. Replacing that estimate with the exact joint probability (11) is the change that matters; the stopping rule above it is a refinement whose separate contribution this study does not establish (§11).

11 Which mechanisms matter

Every ablation plays the same deals against the same panel as the reference configuration, so each row is a matched comparison and the interval is a paired bootstrap over deals.

Table 9: Paired ablations. Positive "full − ablated" means the complete FishBot v0.4 won more often than the modified policy on identical deals.

Change	Win rate	Full − ablated	95% paired CI
FishBot v0.4 (reference)	77.50%	—	—
Drop capacity conditioning as well (C4)	21.48%	+56.03	+54.27–+57.75
Drop certificates and use plain Sinkhorn	24.73%	+52.78	+50.98–+54.55
Drop ask-legality certificates (C5)	28.60%	+48.90	+47.00–+50.78
Exact block posterior instead of the fast one	71.30%	+6.20	+4.35–+8.05
Fast posterior with the policy prior off (matches block)	73.62%	+3.88	+2.08–+5.73
Remove the fitted soft policy prior	73.62%	+3.88	+2.08–+5.73
Zero the lock-completion weight	74.30%	+3.20	+1.55–+4.85
Zero the hit-probability weight	75.95%	+1.55	-0.25–+3.35
Remove the exact two-ply refinement	76.28%	+1.23	-0.53–+3.05
Declaration threshold 0.99	77.30%	+0.20	-0.20–+0.60
Choose the allocation by conditioned greedy MAP	77.33%	+0.18	-0.12–+0.47
Threshold declarations instead of optimal stopping	77.38%	+0.12	-1.23–+1.47
Zero both information-leak weights	77.38%	+0.12	-1.38–+1.65
Declaration threshold 0.80	77.48%	+0.03	-0.18–+0.22
Cash locked half-suits immediately	77.50%	+0.00	+0.00–+0.00
Remove the one-ply value lookahead	77.83%	-0.33	-1.88–+1.23
Zero the reply-threat weight	77.85%	-0.35	-1.98–+1.27
Zero the runway weight	77.85%	-0.35	-1.65–+0.97

Table 9 separates cleanly into three groups, and the honest summary is that one group dominates and most of the rest is not individually established.

The belief is almost the whole agent. Removing the ask-legality certificates (C5) — keeping exact capacities, exact exclusions, and everything else about the policy — costs more than forty win-rate points. Removing capacity conditioning (C4) as well costs more again. Nothing else in the table is within an order of magnitude of these. This is the same conclusion the v0.3 study reached about ask history, arrived at from the opposite direction: v0.3 fitted an exponential weight on public ask counts and found it decisive; v0.4 replaces the weight with the logical certificate that produces it, and finds the certificate decisive. Fitting v0.3's soft weight *on top of* the certificate is worth nothing measurable, which is the evidence that the certificate subsumes it.

Beyond the belief itself, two terms are separated from zero. The fitted policy prior (14) — the soft weight saying that a player who has taken many turns without asking in a half-suit probably has none of it — is worth a few points, which is a change from the earlier version of this study: an audit found the prior was reaching the declaration estimator but not the ask posterior, and once connected it became measurable. And among the twenty ask features, only lock completion, $\prod_{d \in S \setminus \{c\}} \Pr[d \text{ is ours}]$ — the probability that this one ask converts a contested half-suit into a frozen one — has an interval excluding zero.

Everything else is inside noise, including the raw hit-probability weight. That last result is worth stating plainly rather than hiding: zeroing w_0 barely changes the policy, because the squared-hit term, the certainty indicator, the runway term and the value lookahead all carry the same signal. Correlated features substitute for one another, and a paired design controls the deals, not the collinearity.

Three entries have negative point estimates — removing the reply-threat weight, the runway weight, or the value lookahead each nominally *improves* the policy — and no interval excludes zero. We keep all three, because dropping a component on the strength of a non-significant ablation is selection on noise, and because the shipping configuration was chosen on a validation bank rather than on this one. But we report the sign.

The declaration *probability* matters; the stopping rule on top of it is not separately established. FishBot v0.4 declares at 98.55% accuracy against v0.3's 82.02% on the same games (§10), and that gap is worth a large number of half-suits per game. But the ablations locate the credit in the estimate of the allocation probability — which is part of the belief, and falls under the first paragraph — not in the stopping rule that consumes it. Replacing (15) with a fixed confidence threshold, and moving that threshold from 0.80 to 0.99, both change the win rate by amounts indistinguishable from zero. One entry is exactly zero by construction and is reported for completeness: the "cash locked half-suits immediately" switch is inert whenever the stopping rule is active, because the stopping rule decides first. The locked threshold of Table 2 is inert for the same reason, which is worth knowing before reading a value into it.

So Theorem 2 is a correct and, as far as we can tell, previously unstated fact about the game, and the policy that follows from it declares extremely accurately — but this study does not establish that the optimal-stopping formulation beats a well-calibrated threshold. What it does establish is that both need a good joint probability underneath them.

Exactness makes the numbers right and the play slightly worse. This is the result we least expected. Substituting the exact block posterior for the fast approximation it validates costs measurable win rate, and the effect survives the obvious controls. About two thirds of the gap is the policy prior, which the block engine does not carry — hence the matched row, the fast posterior with that prior switched off — but a residual of two to three points remains after that correction, in the same direction. Repeating the substitution under a deliberately *unfitted* policy (pure posterior-greedy asking, no value lookahead, no two-ply refinement, no prior, so neither belief has been fitted to) reproduces the direction on a smaller margin (`research/v04/results/E12-exactness.txt`). So this is not simply the ask weights having absorbed Sinkhorn's biases.

Two mechanisms are consistent with what we see and we cannot separate them here. The declaration thresholds were fitted against the approximate estimator, and the exact one has a different distribution, so the block variant runs a correctly computed probability into a threshold calibrated for something else. And an ask is chosen by a ranking over roughly forty candidates, which tolerates a great deal of noise in the scores that produce it — the positions where Sinkhorn is worst are sharp, nearly resolved ones, which are exactly the positions where the ranking is not close.

This is a cleaner instance of a phenomenon Rebstock et al. report in Skat, where a *cheating* inference module played worse than an ordinary one: better beliefs do not monotonically produce better play, because the policy above them is fitted rather than optimal. Fish is an unusually good place to observe it, because belief quality here is exactly and independently controllable. The experiment that would settle it — refit the entire policy against the exact posterior and compare the two fitted agents — is the obvious next one, and we have not run it. We ship the fast posterior because it is both stronger here and roughly twenty times cheaper in whole-game throughput, while noting that the exact engine is what licenses every probability statement in this paper.

12 Robustness

12.1 Is there one best policy across playstyles?

This is the question the project set out to answer, and it deserves a direct answer rather than a table. The honest one has two halves, and the useful observation is that they correspond to the two halves of the agent.

The inference half is playstyle-independent, and provably so. Constraints (C1)–(C5) are logical consequences of the rules. "The asker holds another card of that half-suit" is true whether the asker is a world champion, a novice, or a uniform random control; so is "a miss proves the target lacks the card", and so is every capacity identity. Nothing in the exact posterior of §5 is fitted to an opponent, and no opponent can make it wrong. The same is true of Theorem 2: a half-suit held by one team cannot be asked in by the other, whatever the other team's style. So a genuinely universal core exists, and it is the part of FishBot v0.4 that the ablations identify as carrying the most value.

The decision half cannot be universally optimal, and we should not claim it is. Choosing among asks trades off quantities — how dangerous it is to hand this opponent the turn, how much an ask reveals, when to cash a half-suit — whose values depend on what the other five players do next. In game theoretic terms, the best response to a policy is a function of that policy, so no single deterministic policy is simultaneously a best response to all of them. The strongest defensible target is not "best against everyone" but "minimally exploitable", and that is a team-maxmin-with-correlation equilibrium which we do not compute. Our fitting objective is a deliberate approximation of it: we maximise the worst-case win rate over a panel rather than the average ((19)), which is a max-min over a finite, hand-built opponent set rather than over all policies.

What the evidence supports. Within the population we can construct, one policy *is* best against every member simultaneously — FishBot v0.4's worst result against any of the eight is 75.07%, and it is ahead of every one of them. That is the strongest empirical form of the claim available without an exploitability bound. Against a purpose-built adversary the picture changes: an exploiter fitted specifically against it reaches only 51.19% (§12.1 is answered in the affirmative for this population, and §12.4 qualifies it). The two results are not in tension; they say that the agent is uniformly strong against the styles that exist and remains exploitable by a style built to beat it, which is the expected state of affairs for any policy short of an equilibrium.

12.2 Across playstyles

The fitting objective (19) targets the worst opponent rather than the average one, and Table 4 reports the whole profile rather than a row average. Two of the policies in that table — the misdirection artist and the random control — were deliberately excluded from the fitting panel and are therefore held-out opponents in the strict sense.

12.3 Across rule dialects

Because the v0.4 engine implements the rule differences of §2 as switches, the same frozen policy can be replayed under other dialects. Table 10 reports three: the v0.3 dialect in which declarations are restricted to the declarer's own turn; the 48-card, eight-half-suit Literature form; and the full rules with out-of-turn declarations disabled but everything else intact.

Table 10: The frozen FishBot v0.4 configuration replayed under other rule dialects.

Rule dialect	Opponent	Win rate	95% CI
v0.3 dialect (turn-only declarations)	FishBot v0.3	64.46%	62.6–66.4
48-card, eight half-suits	FishBot v0.3	72.54%	70.8–74.2
No out-of-turn declarations	FishBot v0.3	67.71%	65.9–69.5
v0.3 dialect (turn-only declarations)	Turn-starvation lockout	77.25%	75.6–78.9
48-card, eight half-suits	Turn-starvation lockout	74.79%	73.2–76.4
No out-of-turn declarations	Turn-starvation lockout	80.17%	78.5–81.8
v0.3 dialect (turn-only declarations)	Posterior detective	76.21%	74.4–78.0
48-card, eight half-suits	Posterior detective	72.21%	70.5–73.9
No out-of-turn declarations	Posterior detective	74.75%	73.0–76.5

12.4 Against an opponent that has studied it

Head-to-head results against a fixed population say nothing about how a well-informed adversary would fare. Because exact best-response computation is out of reach at this game size, we estimate a *local* best response: we freeze FishBot v0.4 and run the same fitting machinery with that frozen policy as the entire opponent panel, so the optimiser's only objective is to beat it. The resulting win rate is an estimate of how much an adversary who can study the bot and search the same policy class can extract. We repeat the procedure against FishBot v0.3 for comparison, since an exploitability number is only interpretable relative to something.

The outcome is the most interesting single result in this paper. An adversary fitted specifically against the frozen FishBot v0.4, inside the same policy class and with the same optimiser and budget, reaches 51.19% on a bank neither the exploiter nor the target had seen — an interval that straddles even money, so the probe simply fails to establish any advantage over its target. The identical procedure applied to FishBot v0.3 reaches 77.31%.

The gap is what matters. A fixed opponent pool cannot distinguish a policy that is strong from one that is strong *and hard to counter*, and by this measure the two predecessors are very different objects: v0.3 is comfortably exploitable by a purpose-built adversary, and v0.4, within the reach of this probe, is not.

The precise reading is worth stating, because it is stronger than "hard to beat". The exploiter searches the same parametric family that FishBot v0.4 belongs to, with the same optimiser and budget, and its objective is solely to beat v0.4. That it converges to parity means no member of that family, found by that search, is a profitable deviation — which is the empirical signature of an approximate symmetric

equilibrium *of the restricted game* in which both teams are confined to this policy class. It is emphatically not an equilibrium of Fish: the class is a twenty-feature linear score with a dozen decision knobs, and the true strategy space is vastly larger. But it does say that the fitting procedure has run out of room against itself, which is the point at which a larger policy class, not more fitting, is what the next version needs.

Three caveats keep this from being an exploitability bound. The probe searches one parametric policy class with one optimiser for a fixed budget, so it is a lower bound and a differently-shaped adversary could do better. The interval straddles 50%, so "cannot beat it" is a failure to establish an advantage rather than a demonstrated absence of one. And a policy playing a near-copy of itself is a mirror match, which is precisely where §6.3's deadlock pressure bites hardest; the residual rate at which that probe's games reach the action cap is reported in the artifact and is materially higher than anywhere else in this study.

Table 11: Local best-response probe. Each row fits a fresh policy, inside the same policy class and with the same optimiser, whose only objective is to beat the frozen policy named. The win rate is what that adversary achieved; lower is better for the frozen policy.

Frozen policy	Exploiter win rate	95% CI	Mean sets conceded
FishBot v0.4	51.19%	49.7–52.7	4.550
FishBot v0.3	77.31%	76.0–78.7	5.547
Posterior detective	76.47%	75.1–77.8	5.471

Table 11 gives the three probes. The probe is a lower bound on exploitability, not an upper one: a better optimiser, a longer budget, or a policy class outside the one searched could all do better. It is reported because a head-to-head result against a fixed population says nothing at all about an adversary who has studied the agent, and because an exploitability figure is only interpretable next to another one.

13 What a human should take from this

FishBot v0.4 maintains a 54×6 posterior and evaluates a hundred-odd candidate asks per turn. A person cannot. But the mechanisms the ablations identify as decisive are not the expensive ones, and three of them are cheap enough to run at a table.

13.1 The deduction that pays for itself

An ask is a confession. When somebody asks for the queen of hearts, they have told the table two things: they do not hold the queen, and they *do* hold at least one other high heart. The second is the one people forget, and together with card counting it is where essentially all of the agent's strength lives: strip either from FishBot and it drops from winning roughly three games in four to losing three in four. It applies to teammates as much as to opponents, and it does not expire: the cards it refers to cannot move except by a public ask.

A miss is a second confession. It rules the named card out of the target permanently, for the same reason.

Keep these as exclusions, not impressions. "Somebody wanted low clubs" is worth little; "the 7 of clubs is not with Iris, not with Theo, and Sage holds one of the other five" is worth a great deal, because it combines with counting.

13.2 Counting closes the deductions

Card counts are public, and they are the constraint that turns partial exclusions into certainties. If a player has four cards and you have located four of them, they hold nothing else — every unresolved card is excluded from them at once. Run this after every transfer, on the two players whose counts changed. Most of FishBot's "impossible" deductions come from nothing more sophisticated than this.

13.3 The claim you should not make

If your team holds all six cards of a half-suit, nobody can take it. An opponent needs a card of a half-suit to ask in it, and they have none. The set is frozen until it is claimed (Theorem 2), and if an opponent claims it they must be wrong, which hands it to you. It follows that claiming early buys you no safety — and costs you a little, because the six cards leaving play sharpens everyone's count of what remains. So:

Two qualifications, because the simulator does not simply endorse patience. The fitted agent is in fact the *less* patient of the two policies we measured (§6), and the "cash locked half-suits immediately" switch is inert in the shipped configuration, so no experiment here isolates the value of waiting. What the rules do guarantee is the safety, not the profit.

- If you are sure your side has all six but unsure which teammate has what, you are under no time pressure — nobody can take it. Use that freedom to resolve the split rather than guessing, but do not sit on it out of principle: the cards buy you nothing at the table, and the point only scores once you claim.
- If a half-suit is *not* yet fully yours, the calculus reverses: waiting risks an opponent asking one of your cards away, so a confident claim is right.
- Do not sit on a hand that is entirely locked cards. Those cards license only asks that are certain to fail, so your turn is worth nothing. Cash the half-suit, empty your hand, and hand the turn to a teammate who can use it.

13.4 Choosing the ask

Rank candidates roughly in this order, which is what the fitted weights do:

1. **Certainty first.** A card you have located is free: you take it and you keep the turn. Take every one of these before anything speculative.
2. **Then the run, not the card.** Because a hit keeps the turn, the value of an ask is the length of the sequence it starts. Prefer a likely card that leaves another likely card behind it over an equally likely card that leaves nothing.
3. **Then the lock.** An ask that would complete your team's ownership of a half-suit is worth more than its hit probability suggests, because it converts a contested set into a frozen one.
4. **Then, who receives the miss.** You are choosing your successor as much as your card. Avoid handing the turn to the player who is visibly loaded in a half-suit where your side is exposed.
5. **And what you are giving away.** Your first ask in a half-suit announces that you hold one of its cards. If you are choosing between two near-equal asks, prefer the one in a half-suit you have already revealed.

13.5 Declaring, concretely

Before you claim, say the six cards and a name for each, out loud in your head. Separate "our side has it" from "I know which of us has it" — they are different claims and only the second is enough. If either is shaky and the half-suit is already locked, wait; if it is not locked, weigh the claim against the chance an opponent takes a card first.

13.6 The endgame nobody plans for

When one team runs out of cards, the other must claim every remaining half-suit with no more asking. Two consequences follow that most players get wrong.

First, **running out of cards is not a defeat.** A cardless player cannot be asked, so nothing can be taken from them, and a cardless *team* forces the opponents to declare everything — which they will sometimes get wrong, handing you the set. Both of these are consequences of the rules rather than findings of this study: we did not isolate the value of emptying a hand deliberately, and we flag it as an option worth knowing rather than a measured recommendation.

Second, when it is your team that must claim everything, **claim your surest half-suit first.** A correct claim announces the exact split of six cards, which changes everyone's counts and can settle the half-suit you were unsure of. Order matters, and the safe claim first is the right order.

13.7 What the simulator does not support

The v0.3 study found no reliable benefit from deliberate misdirection, and nothing in this study changes that. FishBot v0.4 has no bluffing rule and beats the misdirection archetype essentially always. Theatre that does not change what you know or who holds the turn is not worth paying for.

14 Limitations and threats to validity

- **The posterior is exact with respect to the rules, not to the opponents.** Constraints (C1)–(C5) are logical consequences of the rules and hold against any opponent whatsoever. The uniform prior over deals consistent with them is not, however, the full Bayesian posterior: that would additionally weight each deal by how likely the observed actions are under the opponents' actual policies, which is playstyle dependent. We implement and fit the natural correction (14) and find it adds nothing once the certificates are enforced, but that is evidence about this opponent population, not a theorem.
- **Population dependence.** Being best in this population is not being unexploitable. Our local best-response probe searches one policy class with one optimiser; a differently-shaped adversary could do better.
- **No equilibrium claim.** Fish with three-player teams and no communication channel calls for a team-maxmin equilibrium with correlation, not a Nash equilibrium, and we compute neither. Nothing here bounds exploitability.
- **Fitted on simulated opponents.** There are no expert human game logs and no independently authored Fish engines to test against. Every opponent in this study was written by us or ported from v0.3, which is a real limit on external validity.
- **The value function is linear.** Ridge-fitted on 405,000 self-play decision points it reaches $R^2 = 0.29$ against the final half-suit differential (`research/v04/results/E14-valuefit-stats.txt`), which

caps how much the one-ply lookahead and the stopping rule built on it can extract, and is consistent with the ablations being unable to separate either from zero.

- **Termination is imposed, not derived.** Theorem 2 makes patience correct, and §6.3 shows that correct patience on both sides deadlocks. Our escalation rule terminates every game in this study, but its horizons are hand-set constants, not the output of any analysis, and a differently-tuned pair of policies could still find a cycle inside them.
- **The exploitability probe is a lower bound.** It searches one parametric policy class with one optimiser for a fixed budget. That it fails to beat FishBot v0.4 is evidence, not a bound; a differently-shaped adversary, or a larger budget, could succeed.
- **Ablation interaction.** Features are correlated; a term can look unnecessary because a correlated term substitutes for it. The paired design controls the deals, not the collinearity.
- **Rule dialect.** Results are for the 54-card, nine-half-suit form with wrong declarations awarded to the opponents and no prearranged conventions. §12 measures sensitivity to some of these, not all.

15 Reproducibility

The engine is a single C++20 translation unit in `engine/src`; the policy is `engine/src/v04.hpp`, the belief engines are `engine/src/belief.hpp` (constraints, fast posterior) and `engine/src/blockdp.hpp` (exact posterior), the v0.3 population is ported in `engine/src/baselines.hpp`, and the game driver is `engine/src/game.hpp`. Build with `make` in `engine/`.

```
./fish verify   --games=600            # rules, information safety, belief soundness
./fish selftest --games=40             # exact vs exact vs sampled posteriors
./fish match    --a=v04 --b=v03 --games=700 --rotations=6 --seed=90210
./fish matrix   --policies=v04,v03,... --games=200 --rotations=6
./fish ablate   --ref=v04 --variants="..." --panel=v03,lockout,detective,v02
./fish calibrate --a=v04 --b=v03 --games=600
./fish fitvalue --a=v04 --b=v04 --games=400      # ridge-fit the value function
./fish tune     --panel=... --full --gens=42     # cross-entropy policy fitting
./experiments.sh                                 # the whole battery
```

Every policy is a string: `v04:belief=block,decl=0.95,topk=6` constructs the exact-posterior variant with those settings, and `allparams=` accepts the full fitted vector, so every configuration in every table can be reconstructed from the artifact files. Results land in `research/v04/results/` as JSON; `engine/build_tables.py` turns them into the tables of this paper. Fitting traces are in `research/v04/runs/`, and the literature survey the design was drawn from is in `research/v04/lit/`.

16 Conclusion

Three things changed between FishBot v0.3 and v0.4, and they are worth separating because they are of different kinds.

The first is a modelling correction. The simulator now implements the rules as written — declarations at any moment, a chosen successor for a cardless player's turn, and an endgame in which teammates exchange only willingness — and the corrections are switches, so their effect is measured rather than assumed.

The second is a structural observation with a computational payoff. Because every card movement in Fish is public, the hidden state is exactly the initial deal. That makes the posterior a degree-constrained

counting problem whose layer index is determined by its own state, so exact marginals, exact allocation probabilities and rejection-free sampling all cost a few hundred thousand multiply–adds; and it makes the one disjunctive constraint — the certificate that an ask carries — tractable too, because every certificate is confined to a single half-suit and can be enumerated inside it. The ablations say that certificate is the most valuable single thing the agent knows.

The third is a strategic result that follows from one line of the rules. A half-suit held entirely by one team cannot be asked in by the other, so it cannot be taken back; waiting on it is free, and the only real costs of holding it are positional. Declaration therefore stops being a confidence threshold and becomes an optimal-stopping problem, which is the decision no previous Fish agent modelled and which the ablations show is worth real win rate.

A fourth thing emerged that we were not looking for. Because a half-suit frozen by Theorem 2 also admits no further information about itself, two policies that both correctly decline to cash an uncertain one deadlock forever; we measured a fitted configuration failing to terminate in a fifth of its self-play deals at any action cap we tried. Termination in Fish is a property of the policies, not of the rules, and a tournament form of the game needs an explicit forced-claims provision.

What we have not done is bound exploitability, compute a team equilibrium, or test against a human expert or an independently written engine. FishBot v0.4 is the strongest policy in the population we can construct and it is calibrated, audited and reproducible; it is not a solved game, and the distance between those two claims is where the next version belongs.

References

[1] FishLab Research Project, “FishBot v0.3: A Count-Conditioned, Empirically Tuned Policy for Six-Player Canadian Fish,” 2026.

[2] J. McLeod, “Literature,” Pagat, © 2006–2025, last updated July 1, 2026.
https://www.pagat.com/quartet/literature.html

[3] M. Develin, “Canadian Fish,” chapter 9 of his card-games manual.
https://web.archive.org/web/20170720075756/http://bantha.org/~develin/cardgames.html#ch9

[4] Deposit Genius, “Literature Game Strategy (Canadian Fish).”
https://depositgenius.com/literature-strategy-canadian-fish/

[5] Canadian Fish Project, “Strategy notes,” 2026. (The public page was found to carry no strategy content when the review corpus fetched it; cited for continuity with the v0.3 paper.)
https://canadian-fish.vercel.app/strategy

[6] Wikipedia, “Literature (card game).”
https://en.wikipedia.org/wiki/Literature_(card_game)

[7] N. Somani, “literature: card game implementation and learning bot,” GitHub repository (MIT licence, Python), 2018–.
https://github.com/neelsomani/literature

[8] N. Somani, “literature-server,” GitHub repository; live deployment at https://literature.neelsomani.com/
https://github.com/neelsomani/literature-server

[9] "Literature: Fish Card Game" (`com.cards.game.literature`), Google Play store listing. Closed source; the advertised bot methodology is unverified.
https://play.google.com/store/apps/details?id=com.cards.game.literature

[10] D. Zha, K.-H. Lai, S. Huang, Y. Cao, K. Reddy, J. Vargas, R. Wei, J. Guo, and X. Hu, "RLCard: A Toolkit for Reinforcement Learning in Card Games," arXiv:1910.04376, 2019.
https://arxiv.org/abs/1910.04376

[11] D. Levy, "The Million Pound Bridge Program," *Heuristic Programming in Artificial Intelligence: The First Computer Olympiad*, Ellis Horwood, 1989. (Cited in the review corpus at second hand; primary text unverified.)

[12] M. L. Ginsberg, "GIB: Imperfect Information in a Computationally Challenging Game," *Journal of Artificial Intelligence Research* 14:303–358, 2001.
https://www.jair.org/index.php/jair/article/view/10279

[13] I. Frank and D. Basin, "Search in games with incomplete information: a case study using Bridge card play," *Artificial Intelligence* 100(1–2):87–123, 1998.

[14] I. Frank, D. Basin, and H. Matsubara, "Finding Optimal Strategies for Imperfect Information Games," *AAAI-98*, pp. 500–507.
https://cdn.aaai.org/AAAI/1998/AAAI98-071.pdf

[15] J. Long, N. R. Sturtevant, M. Buro, and T. Furtak, "Understanding the Success of Perfect Information Monte Carlo Sampling in Game Tree Search," *AAAI-10*, pp. 134–140.
https://webdocs.cs.ualberta.ca/~nathanst/papers/pimc.pdf

[16] R. Pavlicek, "Dealing with Constraints," Bridge with Pavlicek.
https://www.rpbridge.net/8h11.htm

[17] P. I. Cowling, E. J. Powley, and D. Whitehouse, "Information Set Monte Carlo Tree Search," *IEEE Transactions on Computational Intelligence and AI in Games* 4(2):120–143, 2012.
https://eprints.whiterose.ac.uk/id/eprint/75048/1/CowlingPowleyWhitehouse2012.pdf

[18] P. I. Cowling, D. Whitehouse, and E. J. Powley, "Emergent Bluffing and Inference with Monte Carlo Tree Search," *IEEE CIG 2015*, pp. 114–121.
http://orangehelicopter.com/academic/papers/cig15.pdf

[19] D. Whitehouse, P. I. Cowling, E. J. Powley, and J. Rollason, "Integrating MCTS with Knowledge-Based Methods to Create Engaging Play in a Commercial Mobile Game," *AIIDE 2013*.
https://cdn.aaai.org/ojs/12679/12679-52-16196-1-2-20201228.pdf

[20] J. Goodman, "Re-determinizing Information Set Monte Carlo Tree Search in Hanabi," arXiv:1902.06075, 2019.
https://arxiv.org/abs/1902.06075

[21] V. Lisý, M. Lanctot, and M. Bowling, "Online Monte Carlo Counterfactual Regret Minimization for Search in Imperfect Information Games," *AAMAS 2015*, pp. 27–36.
https://mlanctot.info/files/papers/aamas15-iioos.pdf

[22] I. Zuckerman, A. Felner, and S. Kraus, "Mixing Search Strategies for Multi-Player Games," *IJCAI-09*, pp. 646–651.
https://www.ijcai.org/Proceedings/09/Papers/113.pdf

[23] M. Buro, J. R. Long, T. Furtak, and N. R. Sturtevant, "Improving State Evaluation, Inference, and Search in Trick-Based Card Games," *IJCAI-09*, pp. 1407–1413.
https://www.ijcai.org/Proceedings/09/Papers/236.pdf

[24] C. Solinas, D. Rebstock, and M. Buro, "Improving Search with Supervised Learning in Trick-Based Card Games," *AAAI-19*; arXiv:1903.09604.
https://arxiv.org/abs/1903.09604

[25] D. Rebstock, C. Solinas, M. Buro, and N. R. Sturtevant, "Policy Based Inference in Trick-Taking Card Games," *IEEE CoG 2019*; arXiv:1905.10911.
https://arxiv.org/abs/1905.10911

[26] C. Solinas, D. Rebstock, N. R. Sturtevant, and M. Buro, "History Filtering in Imperfect Information Games: Algorithms and Complexity," *NeurIPS 2023*; arXiv:2311.14651.
https://arxiv.org/pdf/2311.14651

[27] M. Zinkevich, M. Johanson, M. Bowling, and C. Piccione, "Regret Minimization in Games with Incomplete Information," *NIPS 2007*.

[28] M. Lanctot, K. Waugh, M. Zinkevich, and M. Bowling, "Monte Carlo Sampling for Regret Minimization in Extensive Games," *NIPS 2009*.

[29] O. Tammelin, "Solving Large Imperfect Information Games Using CFR+," arXiv:1407.5042, 2014.
https://arxiv.org/abs/1407.5042

[30] N. Brown and T. Sandholm, "Solving Imperfect-Information Games via Discounted Regret Minimization," *AAAI 2019*; arXiv:1809.04040.
https://arxiv.org/abs/1809.04040

[31] N. Brown, A. Lerer, S. Gross, and T. Sandholm, "Deep Counterfactual Regret Minimization," *ICML 2019*; arXiv:1811.00164.
https://arxiv.org/abs/1811.00164

[32] E. Steinberger, A. Lerer, and N. Brown, "DREAM: Deep Regret Minimization with Advantage Baselines and Model-Free Learning," arXiv:2006.10410, 2020.
https://arxiv.org/abs/2006.10410

[33] N. Brown, A. Bakhtin, A. Lerer, and Q. Gong, "Combining Deep Reinforcement Learning and Search for Imperfect-Information Games," *NeurIPS 2020*; arXiv:2007.13544.
https://arxiv.org/abs/2007.13544

[34] M. Schmid, M. Moravík, N. Burch, R. Kadlec, J. Davidson, K. Waugh, N. Bard, F. Timbers, M. Lanctot, Z. Holland, E. Davoodi, A. Christianson, and M. Bowling, "Student of Games: A unified learning algorithm for both perfect and imperfect information games," *Science Advances* 9(46), 2023; arXiv:2112.03178.
https://arxiv.org/abs/2112.03178

[35] A. Nayyar, A. Mahajan, and D. Teneketzis, "Decentralized Stochastic Control with Partial History Sharing: A Common Information Approach," *IEEE Transactions on Automatic Control*, 2013. (Cited at second hand in the review corpus; bibliographic details unverified.)

[36] A. Celli and N. Gatti, "Computational Results for Extensive-Form Adversarial Team Games," *AAAI 2018*; arXiv:1711.06930.
https://arxiv.org/abs/1711.06930

[37] G. Farina, A. Celli, N. Gatti, and T. Sandholm, "Connecting Optimal Ex-Ante Collusion in Teams to Extensive-Form Correlation: Faster Algorithms and Positive Complexity Results," *ICML 2021*, PMLR 139:3164–3173. (Read at abstract level in the review corpus.)

[38] B. H. Zhang, G. Farina, A. Celli, and T. Sandholm, "Team Belief DAG: Generalizing the Sequence Form to Team Games for Fast Computation of Correlated Team Max-Min Equilibria via Regret Minimization," arXiv:2202.00789, 2022 (ICML 2023).
https://arxiv.org/abs/2202.00789

[39] L. Carminati, F. Cacciamani, M. Ciccone, and N. Gatti, "A Marriage between Adversarial Team Games and 2-player Games: Enabling Abstractions, No-regret Learning and Subgame Solving," *ICML 2022*; arXiv:2206.09161.
https://arxiv.org/abs/2206.09161

[40] S. McAleer, G. Farina, G. Zhou, M. Wang, Y. Yang, and T. Sandholm, "Team-PSRO for Learning Approximate TMECor in Large Team Games via Cooperative Reinforcement Learning," *NeurIPS 2023*.

[41] Z. Xu, Y. Liang, C. Yu, Y. Wang, and Y. Wu, "Fictitious Cross-Play: Learning Global Nash Equilibrium in Mixed Cooperative-Competitive Games," *AAMAS 2023*; arXiv:2310.03354.
https://arxiv.org/abs/2310.03354

[42] N. Bard, J. N. Foerster, S. Chandar, N. Burch, M. Lanctot, H. F. Song, E. Parisotto, V. Dumoulin, S. Moitra, E. Hughes, I. Dunning, S. Mourad, H. Larochelle, M. G. Bellemare, and M. Bowling, "The Hanabi Challenge: A New Frontier for AI Research," *Artificial Intelligence* 280, 2020; arXiv:1902.00506.
https://arxiv.org/abs/1902.00506

[43] J. N. Foerster, F. Song, E. Hughes, N. Burch, I. Dunning, S. Whiteson, M. Botvinick, and M. Bowling, "Bayesian Action Decoder for Deep Multi-Agent Reinforcement Learning," *ICML 2019*; arXiv:1811.01458.
https://arxiv.org/abs/1811.01458

[44] H. Hu and J. N. Foerster, "Simplified Action Decoder for Deep Multi-Agent Reinforcement Learning," *ICLR 2020*; arXiv:1912.02288.
https://arxiv.org/abs/1912.02288

[45] H. Hu, A. Lerer, A. Peysakhovich, and J. N. Foerster, "'Other-Play' for Zero-Shot Coordination," *ICML 2020*; arXiv:2003.02979.
https://arxiv.org/abs/2003.02979

[46] H. Hu, A. Lerer, B. Cui, L. Pineda, N. Brown, and J. N. Foerster, "Off-Belief Learning," *ICML 2021*; arXiv:2103.04000.
https://arxiv.org/abs/2103.04000

[47] A. Lerer, H. Hu, J. N. Foerster, and N. Brown, "Improving Policies via Search in Cooperative Partially Observable Games," *AAAI 2020*; arXiv:1912.02318.
https://arxiv.org/abs/1912.02318

[48] C. Cox, J. De Silva, P. DeOrsey, F. H. J. Kenter, T. Retter, and J. Tobin, "How to Make the Perfect Fireworks Display: Two Strategies for Hanabi," *Mathematics Magazine* 88(5):323–336, 2015.

[49] J. Farrell and R. Gibbons, "Cheap Talk with Two Audiences," *American Economic Review* 79(5):1214–1223, 1989.

[50] A. D. Wyner, “The Wire-Tap Channel,” *Bell System Technical Journal* 54(8):1355–1387, 1975.

[51] I. Csiszár and J. Körner, “Broadcast Channels with Confidential Messages,” *IEEE Transactions on Information Theory* 24(3):339–348, 1978. (Cited at second hand in the review corpus; primary text unverified.)

[52] R. Lowe, J. N. Foerster, Y.-L. Boureau, J. Pineau, and Y. Dauphin, “On the Pitfalls of Measuring Emergent Communication,” *AAMAS 2019*; arXiv:1903.05168.
https://arxiv.org/abs/1903.05168

[53] J. Rong, T. Qin, and B. An, “Competitive Bridge Bidding with Deep Neural Networks,” *AAMAS 2019*; arXiv:1903.00900.
https://arxiv.org/abs/1903.00900

[54] Y. Tian, Q. Gong, and T. Jiang, “Joint Policy Search for Multi-agent Collaboration with Imperfect Information,” *NeurIPS 2020*; arXiv:2008.06495.
https://arxiv.org/abs/2008.06495

[55] E. Lockhart, N. Burch, N. Bard, S. Borgeaud, T. Eccles, L. Smaira, and R. Smith, “Human-Agent Cooperation in Bridge Bidding,” arXiv:2011.14124, 2020.
https://arxiv.org/abs/2011.14124

[56] H. J. Ryser, *Combinatorial Mathematics*, Carus Mathematical Monographs 14, Mathematical Association of America, 1963. (Cited at second hand in the review corpus.)

[57] D. G. Glynn, “The permanent of a square matrix,” *European Journal of Combinatorics* 31(7):1887–1891, 2010. (Cited at second hand in the review corpus.)

[58] M. Jerrum, A. Sinclair, and E. Vigoda, “A polynomial-time approximation algorithm for the permanent of a matrix with non-negative entries,” *Journal of the ACM* 51(4):671–697, 2004.
https://faculty.cc.gatech.edu/~vigoda/Permanent.pdf

[59] A. Newman and M. Vardi, “FPRAS Approximation of the Matrix Permanent in Practice,” arXiv:2012.03367, 2020.
https://arxiv.org/abs/2012.03367

[60] N. Linial, A. Samorodnitsky, and A. Wigderson, “A deterministic strongly polynomial algorithm for matrix scaling and approximate permanents,” *Combinatorica* 20(4):545–568, 2000.
https://www.math.ias.edu/~avi/PUBLICATIONS/MYPAPERS/LSW98/lsw00.pdf

[61] Y. Chen, P. Diaconis, S. P. Holmes, and J. S. Liu, “Sequential Monte Carlo methods for statistical analysis of tables,” *Journal of the American Statistical Association* 100(469):109–120, 2005.

[62] I. Bezáková, A. Sinclair, D. Štefankovič, and E. Vigoda, “Negative examples for sequential importance sampling of binary contingency tables,” *Algorithmica* 64:606–620 (ESA 2006).
https://people.eecs.berkeley.edu/~sinclair/sis_jasa.pdf

[63] M. Cryan and M. Dyer, “A polynomial-time algorithm to approximately count contingency tables when the number of rows is constant,” *Journal of Computer and System Sciences* 67(2):291–310, 2003; and M. Cryan, M. Dyer, L. A. Goldberg, M. Jerrum, and R. Martin, *SIAM Journal on Computing* 36(1):247–278, 2006.
https://webspace.maths.qmul.ac.uk/m.jerrum/papers/CDGJM06.pdf

[64] N. Bard, J. Hawkin, J. Rubin, and M. Zinkevich, "The Annual Computer Poker Competition," *AI Magazine* 34(2):112–118, 2013; and the competition rules describing duplicate matches and common seeds. (Duplicate variance figures only partially verified in the review corpus.)

[65] M. Zinkevich, M. Bowling, N. Bard, M. Kan, and D. Billings, "Optimal Unbiased Estimators for Evaluating Agent Performance," *AAAI-06*, pp. 573–578. (Primary text unverified in the review corpus.)

[66] M. Bowling, M. Johanson, N. Burch, and D. Szafron, "Strategy Evaluation in Extensive Games with Importance Sampling," *ICML 2008*.
https://poker.cs.ualberta.ca/publications/ICML08.pdf

[67] M. White and M. Bowling, "Learning a Value Analysis Tool for Agent Evaluation," *IJCAI-09*, pp. 1976–1981.
https://www.ijcai.org/Proceedings/09/Papers/326.pdf

[68] N. Burch, M. Schmid, M. Moravík, D. Morrill, and M. Bowling, "AIVAT: A New Variance Reduction Technique for Agent Evaluation in Imperfect Information Games," *AAAI-18*; arXiv:1612.06915.
https://arxiv.org/abs/1612.06915

[69] J. Kim and T. Sandholm, "Heuristic Pathologies and Further Variance Reduction via Uncertainty Propagation in the AIVAT Family of Techniques," arXiv:2605.14261, 2026.
https://arxiv.org/abs/2605.14261

[70] B. Li, Y. Chen, and L. Huang, "AV-AIVAT: 74× Cheaper Agent Evaluation with Certified Anytime-Valid Stopping in Imperfect-Information Games," arXiv:2608.06362, 2026.
https://arxiv.org/abs/2608.06362

[71] V. Lisý and M. Bowling, "Equilibrium Approximation Quality of Current No-Limit Poker Bots," arXiv:1612.07547, 2016.
https://arxiv.org/abs/1612.07547

[72] K. Waugh, D. Schnizlein, M. Bowling, and D. Szafron, "Abstraction Pathologies in Extensive Games," *AAMAS 2009*, pp. 781–788.
https://bowlingmh.github.io/papers/09aamas-abstraction.pdf

[73] R. Coulom, "Whole-History Rating: A Bayesian Rating System for Players of Time-Varying Strength," *Computers and Games 2008*.
https://www.remi-coulom.fr/WHR/WHR.pdf

[74] R. Herbrich, T. Minka, and T. Graepel, "TrueSkill™: A Bayesian Skill Rating System," *Advances in Neural Information Processing Systems* 19, 2006. (Team-model formulas flagged for re-verification in the review corpus.)

[75] D. Balduzzi, K. Tuyls, J. Pérolat, and T. Graepel, "Re-evaluating Evaluation," *NeurIPS 2018*; arXiv:1806.02643.
https://arxiv.org/abs/1806.02643

[76] S. Omidshafiei, C. Papadimitriou, G. Piliouras, K. Tuyls, M. Rowland, J.-B. Lespiau, W. M. Czarnecki, M. Lanctot, J. Pérolat, and R. Munos, "α-Rank: Multi-Agent Evaluation by Evolution," *Scientific Reports* 9:9937, 2019.

[77] S. M. Jordan, Y. Chandak, D. Cohen, M. Zhang, and P. S. Thomas, "Evaluating the Performance of Reinforcement Learning Algorithms," *ICML 2020*, PMLR 119:4962–4973.

[78] M. Van den Bergh, “Comments on Normalized Elo,” Fishtest technical note; and Stockfish contributors, “Statistical Methods and Algorithms in Fishtest.”
https://cantate.be/Fishtest/normalized_elo_practical.pdf